

Table of Contents

Blockchain Scalability.....	1
Chapter overview	1
1. Introduction: why scalability matters	2
2. Layer 1 scaling: making the chain itself faster	7
3. Layer 2 scaling: moving work off-chain	12
4. The data availability problem	22
5. Zero-knowledge proof systems for scalability	26
6. Sequencing: ordering, liveness, and MEV	29
7. Interoperability: scaling's second-order problem.....	31
8. The state of the art: a landscape snapshot (mid-2026)	33
9. Security: attacks and defences in scaled systems	35
10. A programmatic demonstration	39
11. Scalability in practice: DeFi and TradFi.....	46
12. Comparative evaluation	52
13. Open research problems.....	55
14. Summary	57

Blockchain Scalability

“The scalability trilemma is not a law of nature; it is a statement about naive blockchain architectures. The entire history of blockchain scaling research is the history of finding clever ways around it.”

Chapter overview

Scalability has been the defining engineering problem of public blockchains since Bitcoin's earliest years, and it remains the axis along which the most consequential protocol research, the largest capital flows, and the most damaging security failures of the past decade have all been organised. This chapter presents a systematic treatment of the problem and its solutions as they stand today. We begin by defining scalability precisely and explaining why decentralised replicated state machines are so difficult to scale, formalising the constraint popularly known as the scalability trilemma. We then survey the two great families of solutions — on-chain (Layer 1) scaling and off-chain (Layer 2) scaling — tracing each technique from its original research proposal to its current production deployment. Rollups, the dominant scaling paradigm on Ethereum, receive extended treatment, including the data availability problem, data availability sampling as deployed in PeerDAS, optimistic and zero-knowledge proof systems, sequencer architectures, and

based and native rollup designs. A complete, runnable Python program is developed step by step to make the core quantitative ideas concrete: state commitment, rollup batch compression, fraud-proof games, and the probabilistic guarantees of data availability sampling. The second half of the chapter examines the security consequences of scaling — bridge exploits, sequencer failures, prover soundness bugs, and upgrade-key compromise — together with the defences the industry has evolved, and analyses scalability requirements in decentralised and traditional finance. We close with a comparative evaluation of all major approaches and a survey of the open research problems that remain unsolved.

By the end of this chapter, you will be able to:

- Define throughput, latency, finality, and decentralisation precisely and explain the trade-offs between them
- Explain why naive parameter increases do not scale blockchains, using a node-resource cost model
- Describe every major Layer 1 scaling technique: parameter tuning, consensus redesign, parallel execution, sharding, and state management
- Explain how payment channels, sidechains, plasma, and rollups differ in their trust models
- Analyse optimistic and zero-knowledge rollups in depth, including fraud-proof games, validity proofs, zkEVM taxonomy, and real-time proving
- Explain the data availability problem, erasure coding, KZG commitments, and data availability sampling, and reason about their security quantitatively
- Evaluate the security of a Layer 2 system using a structured maturity framework
- Analyse historical attacks on scaling infrastructure and the defences deployed against them
- Assess scalability requirements for DeFi and traditional-finance workloads
- Identify the principal open research problems in the field

1. Introduction: why scalability matters

A blockchain is a replicated state machine. Every full node stores the entire state, receives every transaction, re-executes every transaction, and verifies every block. This total replication is precisely what makes a public blockchain trust-minimised: no participant needs to believe anyone else's claim about the state of the ledger, because every participant checks everything independently. It is also precisely what makes a blockchain slow. A network of ten thousand nodes that all re-execute every transaction has, in aggregate, the throughput of a single node — and not a powerful node, but the *weakest node the protocol wishes to keep in the verifying set*.

The consequences of this design were visible early. Bitcoin, with its 1 MB block-size limit (later reworked into a 4 million weight-unit limit by Segregated Witness), settles on the order of 3–7 transactions per second. Ethereum at launch handled roughly 15 transactions per second; even in 2026, with a gas limit that validators have raised to 60 million and a

roadmap targeting an order-of-magnitude more, Layer 1 Ethereum executes only low hundreds of transactions per second. For comparison, a traditional payment network such as Visa averages a few thousand transactions per second globally and has demonstrated capacity above 65,000 TPS in its own published stress tests, and a single commodity database server can process hundreds of thousands of simple key-value updates per second.

The gap matters because demand has repeatedly exceeded supply, with visible economic consequences:

- In late 2017, the CryptoKitties collectible game congested Ethereum so severely that pending-transaction queues grew six-fold and unrelated applications became unusable — the first mainstream demonstration that a single popular application could saturate a world computer.
- During the DeFi expansion of 2020–2021, average Ethereum transaction fees repeatedly exceeded \$20, with fees for complex contract interactions above \$100 at peaks. Entire classes of application — micropayments, gaming, small-value remittances — were priced out of the chain.
- Bitcoin’s own block-size conflict (2015–2017), fought over whether to raise the 1 MB limit, was the most divisive governance episode in the system’s history and ended in a chain split that produced Bitcoin Cash. It established the political lesson that scaling by parameter change is easy to propose and extraordinarily hard to coordinate.

Scalability, then, is not a cosmetic performance concern. It determines which applications are economically viable, who can afford to transact, and ultimately whether public blockchains remain niche settlement systems or become general-purpose financial infrastructure. The rest of this chapter examines how the field has attacked the problem.

1.1 Definitions and metrics

Discussion of scalability is plagued by imprecise vocabulary, so we fix definitions first.

Throughput is the sustained rate at which the system processes transactions, conventionally measured in transactions per second (TPS). TPS is a crude metric — transactions differ enormously in computational weight — so protocol engineers prefer resource-denominated measures such as *gas per second* (Ethereum) or *compute units per second* (Solana). When comparing systems, always ask what a “transaction” is: a simple payment, an ERC-20 transfer, and a complex DeFi interaction can differ by two orders of magnitude in cost.

Latency is the time from transaction submission to first inclusion in a block. Users perceive latency, not throughput.

Finality is the time until a transaction’s inclusion becomes irreversible under the protocol’s security model. Finality may be *probabilistic* (Bitcoin: reversal probability decays exponentially with confirmations, per the analysis in Nakamoto’s original paper), *economic* (Ethereum: reversal of a finalised block requires destruction of at least one-third of staked ether, tens of billions of dollars), or *absolute* within a classical BFT model (Tendermint-

family chains: a committed block is final unless more than one-third of validators equivocate). Latency and finality are frequently conflated in marketing material; a chain may include a transaction in 400 ms but finalise it only minutes later.

Bandwidth, state, and history are the resource dimensions that scaling actually consumes. Every scaling decision ultimately spends one or more of: network bandwidth (propagating transactions and blocks), computation (executing transactions and verifying proofs), RAM and disk I/O (holding and updating state), and storage (retaining history). A useful mental model, due to the Ethereum research community: *a blockchain's capacity is bounded by the resources of the weakest node the protocol wants to keep able to verify the chain.*

Decentralisation, for the purposes of scaling analysis, is the number and independence of parties that can (a) produce blocks and (b) *verify* blocks. Verification is the more important half. A network in which block production is concentrated but any laptop can cheaply verify correctness and any user can force inclusion of a censored transaction retains most of the security properties users care about; a network that only data centres can verify does not, regardless of how many data centres there are. This observation — that we can tolerate specialised block *producers* if verification remains cheap — is the intellectual foundation of proposer–builder separation, fraud proofs, validity proofs, and data availability sampling, and it recurs throughout this chapter.

1.2 The scalability trilemma

The *scalability trilemma*, articulated by Vitalik Buterin around 2017, asserts that simple blockchain designs can achieve at most two of the following three properties:

1. **Scalability** — the chain processes far more transactions than a single ordinary node could.
2. **Decentralisation** — the chain can be verified, and block production meaningfully influenced, by ordinary participants with consumer hardware.
3. **Security** — the chain resists attackers controlling a large fraction (e.g., up to 50%) of the relevant resource (hash power, stake).

The trilemma is not a theorem; it is an empirical regularity about *naïve* designs, and its author has been explicit that its purpose was to motivate architectures that escape it. The three naïve corners are instructive:

- **Big-block chains** (scalability + security, sacrificing decentralisation): simply raise block size or gas limits until demand is met. Node resource requirements climb until only well-capitalised operators can verify the chain, at which point users must trust rather than verify. Bitcoin Cash and early high-throughput EVM sidechains such as BNB Smart Chain occupy this corner.
- **Small-block chains** (decentralisation + security, sacrificing scalability): Bitcoin's post-2017 settlement, deliberately keeping verification cheap enough for a Raspberry Pi at the cost of ~7 TPS.

- **Multi-chain federations** (scalability + decentralisation, sacrificing security): run many independent small chains. Aggregate throughput is high and each chain is cheap to verify, but each chain is secured by only a fraction of total resources, so an attacker can concentrate force on one chain. Early sharding proposals without shared security, and app-chain ecosystems without shared validation, occupy this corner.

Escaping the trilemma requires breaking the assumption that every verifying node re-executes everything. Two technologies accomplish this and together define the modern era of blockchain scaling:

- **Succinct proofs** (SNARKs/STARKs, and interactive fraud proofs as a weaker cousin) let a node verify that a computation was performed correctly *without redoing it*.
- **Data availability sampling** lets a node verify that all data behind a block *was published* without downloading more than a small random sample of it.

With these, a light node holding a few kilobytes can, in principle, verify a chain carrying arbitrarily large throughput — the load on each verifier grows logarithmically (or better) in the load carried by the chain. Nearly everything in this chapter is an elaboration of these two ideas, their engineering costs, and the new attack surfaces they create.

1.3 A node-resource cost model

Before surveying solutions, it is worth making the bottleneck quantitative, because “just raise the gas limit” remains the most common lay proposal. Consider the resources a full node spends per unit time, as a function of chain throughput T (transactions per second):

Resource	Scales as	Dominant cost driver	Practical ceiling (consumer node, c. 2026)
Bandwidth (ingress)	$O(T)$	Receiving all transactions + blocks	~50–100 Mbps sustained is already exclusionary in much of the world
CPU (execution)	$O(T)$	Re-executing every transaction	EVM single-thread execution; signature verification parallelises well
Disk I/O (state)	$O(T \cdot \log S)$	Random reads/writes to a Merkle-tree state of size S	NVMe random-read IOPS; state trie depth
Disk capacity (state)	$O(\int T dt)$	State grows monotonically with usage	Ethereum state: hundreds of GB and growing; archive history: tens of TB
RAM	$O(S)$ ideally	Caching hot state	32–64 GB typical

Three observations follow. First, several costs are *cumulative*: even if the network could momentarily handle $10\times$ throughput, state and history grow without bound, so today’s affordable node becomes tomorrow’s unaffordable one. This is the *state growth problem*, and it is why gas limits are set far below what current hardware could momentarily

sustain. Second, the binding constraint has shifted over time: in 2015 it was CPU; by the early 2020s it was state I/O and bandwidth; by 2026, with flat storage layouts and parallel execution engines in production, bandwidth and state growth dominate. Third — and this is the key strategic insight of the modern roadmap — different *roles* can face different ceilings. If verification can be made exponentially cheaper than execution (via proofs and sampling), the protocol can allow block *builders* to be powerful machines while keeping block *verifiers* on laptops and phones. The trilemma constrains uniform designs, not stratified ones.

1.4 A brief history of the scaling debate

Because the remainder of the chapter is organised by technique rather than by time, the following timeline fixes the historical sequence. Readers should notice the pattern: roughly five years elapse between a technique’s research articulation and its production maturity, and each generation’s *failure* (big blocks, Plasma, execution sharding) supplied the precise problem statement that the next generation solved.

Year	Event	Significance
2008–2010	Bitcoin launch; 1 MB limit added	Throughput cap as spam defence, later a constitution
2015–2017	Bitcoin block-size wars; SegWit; BCH split	Parameter scaling proven politically unscalable
2016	Lightning paper; Elastico (sharding); Croman et al. position paper	Off-chain and sharding research begins in earnest
2017	CryptoKitties congestion; Plasma paper; trilemma articulated	Demand outruns supply publicly for the first time
2018	Arbitrum paper (USENIX Sec.); STARKs; OmniLedger/RapidChain; DA proofs (Al-Bassam et al.)	The modern toolkit appears in research form
2019–2020	Rollup designs mature; LazyLedger (→Celestia); Ethereum abandons execution sharding; rollup-centric roadmap (Oct 2020)	The great pivot: L1 as data + settlement layer
2020–2021	DeFi/NFT booms; \$100+ L1 fees; Optimism/Arbitrum mainnets; Solana mainnet growth	Rollups ship; integrated-chain counter-thesis ships
2022	Bridge catastrophe year (Ronin \$624M, Wormhole \$326M, Nomad \$190M); Optimism infinite-mint disclosure	Scaling’s trust seams become the dominant loss source
2023	Celestia mainnet (modular DA); dYdX exits to app-chain; based-rollup proposal; EIP-4844 finalised	The DA marketplace and sequencing debates open

Mar 2024	Dencun/EIP-4844 : blobs	L2 fees fall 10–100×; rollup era economically viable
2024–2025	Permissionless fault proofs (OP, Arbitrum BoLD); Stage 1 wave; zkVM industrialisation; restaking/EigenDA; Pectra (6/9 blobs)	Rollup security promises finally redeemed in code
Dec 2025	Fusaka/PeerDAS ; BPO forks; Firedancer on Solana mainnet; Monad mainnet	DAS in production; parallel-EVM generation lands
2026	Real-time ZK proving; Glamsterdam (ePBS, BALs, gas-limit path); Alpenglow activation window; Lean Ethereum roadmap; CTV signalling on Bitcoin	Verification-first scaling becomes explicit strategy everywhere

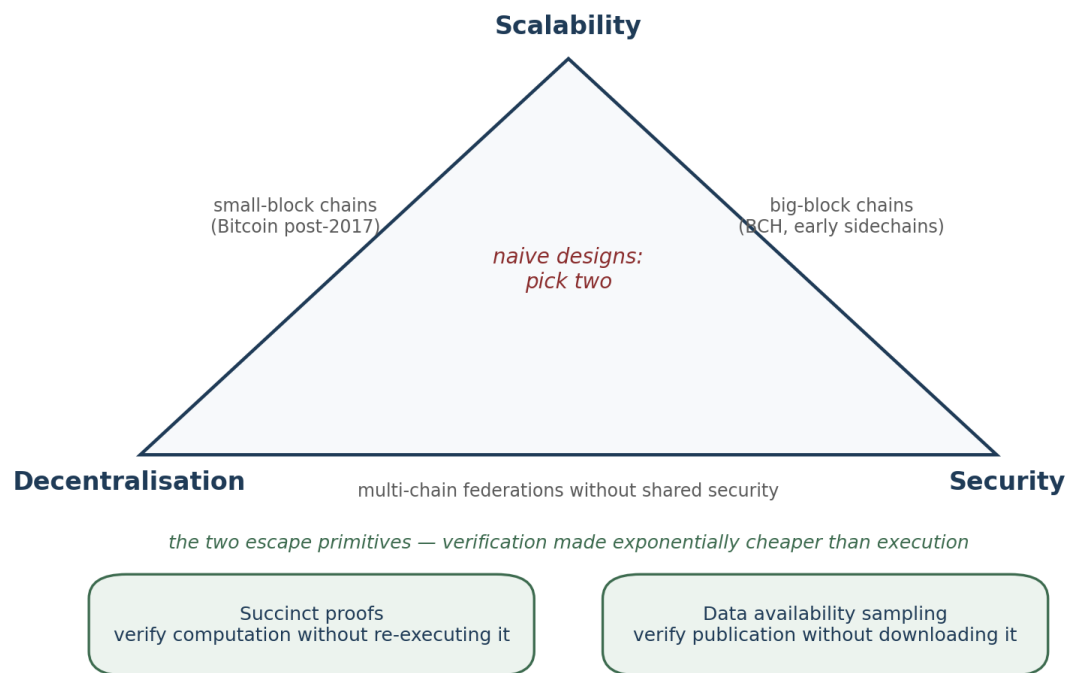


Figure 1.1 — The scalability trilemma and the two escape primitives.

2. Layer 1 scaling: making the chain itself faster

Layer 1 (on-chain) scaling modifies the base protocol so that the chain natively processes more transactions. We examine the five principal families: parameter increases, consensus redesign, parallel execution, sharding, and state management. Each attacks a different term of the cost model above.

2.1 Parameter increases and their limits

The simplest lever is raising block size, block frequency, or the gas limit. It is not useless — Ethereum validators raised the gas limit from 30 million to 36 million, then 45 million, and coordinated around 60 million by early 2026, each step gated on measurements of client performance and propagation times, with EIP-7935/7938-style proposals charting a path toward 200 million alongside the Glamsterdam upgrade. But parameter increases face three hard limits.

Propagation and centralisation. A block must propagate to (nearly) all validators before the next block is produced, or the chain suffers elevated orphan/reorg rates, which disproportionately penalise poorly connected validators and thus centralise block production. Ethereum’s Glamsterdam fork addresses this structurally: *enshrined proposer-builder separation* (ePBS, EIP-7732) splits consensus validation from execution payload propagation, widening the effective propagation window from roughly 2 seconds to roughly 9 seconds within a 12-second slot, and *block-level access lists* (BALs, EIP-7928) ship each block with a declaration of the state it touches so validators can parallelise disk reads and even begin execution optimistically. These two changes exist precisely to make a 3–5× gas-limit increase safe rather than centralising — an instructive example of the modern pattern in which parameter increases are *earned* by prior structural work.

Verification-cost creep. Every doubling of throughput doubles the resources needed to verify, permanently. The Bitcoin block-size wars were, at bottom, a dispute about whether that cost creep was acceptable; the small-block position won on Bitcoin, and its logic — verification must stay cheap — was subsequently absorbed even by high-throughput designs, in the stratified form described in Section 1.3.

State growth. Higher throughput accelerates state growth, degrading I/O performance for all future blocks (Section 2.5).

The honest summary: parameter increases are a legitimate but bounded tool, yielding perhaps one order of magnitude when paired with engineering work, and they are now always deployed as the *last* step after structural changes, never the first.

2.2 Consensus redesign: faster agreement

Consensus is rarely the throughput bottleneck in modern chains — execution and data distribution are — but it dominates *latency and finality*, and poorly designed consensus caps throughput indirectly by forcing conservative block times.

From longest-chain to BFT finality. Nakamoto consensus buys extreme simplicity and permissionless participation at the price of probabilistic finality measured in minutes to hours. Classical BFT protocols (PBFT lineage: Tendermint, HotStuff and its descendants) finalise in seconds with $2f+1$ quorums among a known validator set, and underlie Cosmos-family chains, Aptos, and Sui. Ethereum’s Gasper (Casper-FFG finality gadget over LMD-GHOST fork choice) is a hybrid: blocks every 12 seconds, economic finality after two epochs (~12.8 minutes). The roadmap item *single-slot finality* (SSF) — finalising each block in its own slot using BLS signature aggregation over very large committees — remains one of

Ethereum’s central open engineering projects, now folded into the “Lean Ethereum” consensus track (Section 8).

Case study: Solana’s Alpenglow. Solana historically combined Proof of History (a verifiable delay/clock construction) with Tower BFT voting, giving fast block times (~400 ms) but full finality only after ~32 slots (~12.8 seconds). The Alpenglow redesign — accepted by governance vote in 2025 with mainnet activation targeted for late 2026 — replaces this with a two-phase-commit protocol (components *Votor* for voting and *Rotor* for data dissemination) that finalises in roughly 100–150 milliseconds, about a 100× improvement, by cutting validator round-trips and disseminating blocks via erasure-coded relay. Alongside it, the Firedancer validator client — a from-scratch C/C++ rewrite by Jump Crypto, live on mainnet since December 2025 and running on a growing minority of validators — demonstrated over 1 million TPS in controlled tests, while real-world mainnet throughput sits in the 3,000–10,000 TPS range. The pairing illustrates a general lesson: consensus redesign fixes finality; *client engineering* fixes throughput; and headline laboratory numbers routinely exceed production numbers by two orders of magnitude because production workloads are dominated by state access, not signature checks.

DAG-based ordering. A further family (Narwhal/Bullshark/Mysticeti as used by Sui, and related mempool-DAG designs) decouples data dissemination from ordering: validators continuously gossip batches into a directed acyclic graph, and consensus merely orders the DAG, achieving high throughput and sub-second finality. These designs currently power some of the highest sustained-throughput production L1s.

For orientation, the consensus landscape summarised (approximate production figures, mid-2026):

Protocol family	Block time	Finality	Validator scale	Examples
Nakamoto PoW	~10 min	probabilistic (~60 min for 6 conf.)	permissionless miners	Bitcoin
Gasper (PoS hybrid)	12 s	economic, ~12.8 min (SSF on roadmap)	~1M validators (aggregated)	Ethereum
Tendermint/HotStuff BFT	~1–6 s	absolute, 1 block	~100–200	Cosmos chains, dYdX v4
PoH + Tower BFT → Alpenglow	~400 ms	~12.8 s → ~150 ms	~1,000–1,500	Solana
Mempool-DAG BFT	sub-second	sub-second	~100+	Sui (Mysticeti), Aptos

The table encodes the trade Section 1 predicted: finality speed is purchased with validator-set size and hardware assumptions, which is why Ethereum’s million-validator design finalises in minutes while hundred-validator BFT chains finalise in one block — and why

Ethereum’s single-slot-finality research (large-scale BLS aggregation, validator-set economics) is hard rather than merely unfinished.

2.3 Parallel execution

The EVM as originally specified is sequential: transactions in a block execute one after another against shared state. Yet most transactions touch disjoint state — two unrelated token transfers do not conflict — so parallel execution is the natural next step, borrowing decades of database concurrency-control research.

Two paradigms dominate:

- **Declared access lists (pessimistic).** Transactions declare up front which state they will read and write; the scheduler runs non-conflicting transactions concurrently. Solana’s Sealevel runtime pioneered this on-chain; the FuelVM’s strict UTXO-style access lists are a purer form. The cost is developer burden and the impossibility of dynamic access patterns (a transaction cannot decide what to touch based on what it reads).
- **Optimistic concurrency control (Block-STM).** Execute transactions speculatively in parallel, detect read/write conflicts, and re-execute conflicting transactions. Block-STM, developed for Aptos and since adopted (in variants) by Monad, Sei, Polygon and others, requires no developer declarations and achieves near-linear speed-ups on low-conflict workloads, degrading gracefully toward sequential performance under contention (e.g., everyone trading the same AMM pool).

The 2024–2026 generation of “parallel EVM” chains — Monad (mainnet late 2025, pipelined optimistic execution plus a custom state database, MonadDB, designed for parallel I/O), MegaETH (real-time rollup targeting millisecond blocks with a single powerful sequencer and stateless verification), Sei v2 — demonstrated that EVM-compatible execution at tens of thousands of TPS is achievable *when execution is the only bottleneck*. Ethereum L1 itself takes a more conservative route: BALs (EIP-7928, Glamsterdam) provide the access-list information for parallel *I/O and validation* without changing EVM semantics, and proposals for fully parallel canonical execution remain in research. The enduring caveat for all parallel designs is that real workloads exhibit hot spots — popular AMM pools, oracle updates, mints — which Amdahl’s law converts into much lower effective speed-ups than benchmarks suggest, and which have pushed several designs toward *local fee markets* (Solana’s per-account fee markets being the canonical example) so that contention on one hot account does not price out the rest of the chain.

2.4 Sharding: from execution sharding to data sharding

Sharding — partitioning the network so that different node subsets process different transaction subsets — is the classically “obvious” scaling solution and the one with the longest, most instructive failure-and-pivot history.

The academic lineage. Elastico (Luu et al., CCS 2016) established committee-based sharding with random assignment; OmniLedger (Kokoris-Kogias et al., IEEE S&P 2018) added client-driven atomic cross-shard commits (Atomix) and bias-resistant randomness;

RapidChain (Zamani et al., CCS 2018) improved intra-committee BFT and cross-shard routing, tolerating 1/3 Byzantine participants. These papers defined the canonical problems: secure random committee assignment (to prevent an attacker concentrating on one shard — the *single-shard takeover attack*), cross-shard atomicity, and reshuffling without downtime.

Production execution sharding. NEAR’s Nightshade implements execution sharding in production, treating shard “chunks” as pieces of one logical block, with stateless validation and dynamic resharding added in 2024–2025. Polkadot’s parachain model is sharding under another name: heterogeneous shards (parachains) validated by a randomly assigned subset of relay-chain validators, with *availability-and-validity* checks — erasure-coded distribution of parachain data plus secondary “approval” checks by randomly self-selected validators, escalating to full disputes on disagreement — providing shared security. Polkadot 2.0’s *agile coretime* (2024–2025) replaced slot auctions with a spot/reserved market for validation capacity, reframing the relay chain as a seller of abstract “cores.” The 2026 “JAM” (Join-Accumulate Machine) research direction generalises this further toward a permissionless compute fabric.

Ethereum’s pivot: data sharding. Ethereum’s original roadmap (2017–2020) envisaged 64 execution shards. It was abandoned — a decision worth understanding, since it reshaped the entire industry. Execution sharding fragments composability (cross-shard calls are asynchronous), multiplies protocol complexity (fraud/validity machinery per shard), and — decisively — was rendered unnecessary by rollups: if L2s handle execution, the base layer needs to scale only *data availability*, which is a far easier problem because data can be erasure-coded and sampled (Section 4). The “rollup-centric roadmap” (Buterin, October 2020) formalised the pivot: Ethereum L1 would become a settlement-and-data layer, and “sharding” would come to mean *danksharding* — sharding blob *data* across the validator set while keeping execution unified. Fusaka’s PeerDAS (December 2025) is the first production step of that plan.

The comparative lesson: execution sharding works (NEAR, Polkadot prove it) but pays a permanent complexity and composability tax; data sharding plus rollups achieves similar aggregate scale while keeping each execution environment internally synchronous. The market has overwhelmingly chosen the latter, though the app-chain ecosystems (Cosmos, Polkadot, Avalanche subnets/HyperSDK) remain a real third pole in which sovereignty is valued above shared security.

2.5 State: the quiet bottleneck

Every technique above increases the *rate* at which state is touched and grown, so state management is the load-bearing wall of L1 scaling. Four threads matter.

State representation. Ethereum’s hexary Merkle-Patricia trie makes every state access an $O(\log S)$ chain of hashed disk lookups and makes witnesses (Merkle proofs) large. Modern clients mitigate with flat key-value layouts and snapshots; Monad’s MonadDB and similar engines design the database for parallel access from the start. At the protocol level, Ethereum spent years preparing a move to *Verkle trees* (vector-commitment tries with tiny witnesses), then pivoted in 2025 toward *binary Merkle trees with hash-based proofs*, on the

grounds that Verkle’s elliptic-curve cryptography is not post-quantum secure and that the endgame is ZK-proving every block anyway — hash-based structures are STARK-friendly. This pivot is part of the Lean Ethereum programme (Section 8).

Statelessness. A *stateless client* verifies blocks using witnesses shipped with the block rather than a local state copy, collapsing the disk requirement for verifiers. Weak statelessness (block producers keep state; validators verify witnesses) is the practical target and is a prerequisite for truly light verification of a high-throughput L1.

State expiry and rent. Proposals to expire untouched state (resurrectable via witness) or charge ongoing rent attack the cumulative-growth term directly but face severe backward-compatibility objections; no major general-purpose L1 has shipped them, though Solana’s account-rent deposit model is a partial instance.

History expiry. EIP-4444 and the Portal Network shift ancient history out of the requirement set for full nodes — history, unlike state, need only be *available somewhere*, not held by everyone. Partial history expiry began rolling out on Ethereum in 2025–2026.

2.6 Synthesis: an L1 capacity budget

The L1 techniques compose, and the composition is best seen as a budget exercise. Take Ethereum’s published path as the worked example: today’s ~60M gas per 12 s slot (~5M gas/s) is bounded jointly by worst-case propagation (addressed by ePBS: the payload window widens ~4.5×), worst-case execution and state I/O (addressed by BALs enabling parallel reads and optimistic execution, client-team repricing of mispriced opcodes, and eventually zkEVM verification removing re-execution from validators entirely), state growth (addressed by binary trees, statelessness, and history expiry), and DA bandwidth for rollups (addressed by PeerDAS’s 1/16 custody factor and BPO steps toward 48 blobs). Each remedy unlocks the next parameter increase, which is why the roadmap interleaves structural forks with parameter forks: 60M → ~200M gas around Glamsterdam, gigagas-per-second targets in the Lean Ethereum frame, and blob capacity stepping toward ~0.75 MB/s and beyond. Multiply the terms and Ethereum’s own stated endgame — on the order of 10,000 TPS at L1 and ~1M TPS aggregate with L2s — is not a marketing number but the product of this budget, *conditional* on every line item (real-time proving at home-scale power budgets, DAS at scale under adversarial networking, state design) landing. The conditionality is the point: this chapter’s later sections examine exactly those line items, and Section 13 lists the ones that remain research rather than engineering.

3. Layer 2 scaling: moving work off-chain

Layer 2 approaches accept the base layer’s limits and move execution elsewhere, using the base layer as a court, a bulletin board, or both. The taxonomy is best organised by *what enforces correctness* and *where data lives*. We treat the families in roughly historical order: channels, sidechains, plasma, then rollups — which now dominate — and their data-availability variants.

3.1 Payment and state channels

A payment channel is the oldest L2 idea, present in embryo in Satoshi Nakamoto's nSequence transaction-replacement mechanism and matured through the Spilman (unidirectional) and Poon–Dryja (bidirectional, penalty-based) constructions. Two parties lock funds in a 2-of-2 multisignature output; they then exchange *signed but unbroadcast* transactions that reassign the balance, at arbitrary speed and zero marginal cost; either party may close by broadcasting the latest state. Cheating — broadcasting a stale state — is deterred by a *revocation penalty*: publishing an old state hands the counterparty the entire channel balance after a dispute window. The Lightning Network (Poon & Dryja, 2016) composes channels into a network via Hash Time-Locked Contracts (HTLCs), so payments route atomically across multi-hop paths without trusting intermediaries.

The Poon–Dryja construction rewards close reading, because it is the earliest complete example of the pattern that recurs throughout L2 design: *make cheating possible but unprofitable, and make the base layer the judge*. Each channel state is a pair of asymmetric commitment transactions, one held by each party. Alice's copy pays Bob immediately but pays Alice only after a relative timelock (via `OP_CHECKSEQUENCEVERIFY`), and — crucially — Alice's delayed output can *also* be spent immediately by Bob if he knows the *revocation secret* for that state. When the parties advance to a new state, they exchange the revocation secrets of the old one. Now if Alice ever broadcasts a stale state (one that favours her), she has, by construction, already handed Bob the key that converts her own delayed output into Bob's instant one: the penalty is total and self-enforcing, and the timelock exists precisely to give Bob the window to use it. Multi-hop payments extend the game: each hop is an HTLC — “payable to Bob with the preimage of hash H before time T, refundable to Alice after T” — with timeouts strictly decreasing along the path so that a preimage revealed at the destination can always be pulled back hop-by-hop before any refund matures. Every later fraud-proof system in this chapter is a generalisation of this two-move structure: an optimistic claim, a challenge window, evidence checkable by a dumb but incorruptible arbiter, and incentives arranged so the honest party wins by default.

Channels offer the strongest possible L2 properties on their narrow domain: instant finality between participants, zero per-transaction chain footprint, full self-custody. Their limits are equally structural: capital is locked per-channel and per-direction; both parties must monitor the chain (or delegate to *watchtowers*) to catch stale-state fraud within the dispute window; receiving requires inbound liquidity; and channels serve *payments*, not general shared-state computation (generalised state channels — Perun's virtual channels, Counterfactual — were built and formally verified but never found product-market fit beyond payments).

Lightning in 2026. The network holds roughly 5,000–5,600 BTC of public capacity across ~40,000 channels and ~17,000 nodes, processes on the order of a billion dollars of monthly volume, and gained a major new workload when Tether's USDT went live over Taproot Assets in 2026 — stablecoin transfer being a far larger real-world payments market than BTC transfer itself. Structurally the network has consolidated: fewer, better-capitalised routing nodes, exchange integrations (Coinbase, Binance, OKX) driving volume, and protocol upgrades (splicing for in-place channel resizing, BOLT12 offers for reusable

payment codes) improving operability. Lightning is simultaneously the most successful L2 in payment count and a cautionary tale about operational complexity as a barrier to self-custodial routing at the edge.

Beyond Lightning: shared-UTXO protocols. The newer Bitcoin L2 generation — Ark and Spark most prominently — replaces per-pair channels with *virtual UTXOs* (VTXOs): many users share one on-chain UTXO coordinated by an operator, each holding an off-chain proof of their share and a unilateral exit path. This removes inbound-liquidity and channel-management burdens at the cost of a liveness dependency on the operator and (in Ark’s case) capital-intensive round batching for the operator. These designs would benefit materially from Bitcoin covenant opcodes — most prominently OP_CHECKTEMPLATEVERIFY (BIP 119), whose activation signalling is now formally under way with a 90% miner threshold across 2026–2027 — which would shrink exit footprints and operator capital costs. Bitcoin’s scaling story is thus converging on the same theme as Ethereum’s: commit off-chain activity to compact on-chain commitments and make exits trust-minimised.

3.2 Sidechains

A sidechain is an independent blockchain with its own consensus and validator set, connected to a parent chain by a two-way bridge. The concept dates to the 2014 Blockstream pegged-sidechains paper; production examples include Liquid (federated Bitcoin sidechain), Polygon PoS (historically a checkpointing sidechain, now progressively integrating validity-proof infrastructure), Gnosis Chain, and Ronin.

The critical property: **a sidechain does not inherit the parent chain’s security**. Its safety is bounded by its own consensus (often a small validator or federation set), and its bridge is a separate trusted component. Users of a sidechain trust (a) the sidechain validators not to reorganise or censor and (b) the bridge signers not to steal. The Ronin bridge exploit (March 2022, ~\$624 million, five of nine validator keys compromised by the Lazarus Group via social engineering) is the canonical demonstration of both failure modes at once, and we return to it in Section 9. Sidechains persist because they are simple, cheap, and sovereign; the security-conscious end of the market has migrated to rollups precisely because “trust our validators” proved to be the industry’s single most expensive assumption.

3.3 Plasma: the instructive failure

Plasma (Poon & Buterin, 2017) proposed hierarchical chains committing periodic state roots to Ethereum, with users able to *exit* their funds to L1 by presenting Merkle proofs, and fraudulent exits challengeable during a dispute window. Plasma aimed at rollup-like security without posting transaction data on-chain — and that is exactly where it failed. If a Plasma operator withholds block data, users cannot construct exclusion proofs to challenge fraudulent exits; the only safe response is a *mass exit*, which floods L1 and, in the limit, cannot physically fit within the dispute window. Years of increasingly baroque constructions (Plasma Cash’s per-coin non-fungible histories, Plasma MVP’s exit priority queues) narrowed but never closed the gap for general computation.

Plasma's legacy is the precise identification of the **data availability problem** as *the* fundamental obstacle for off-chain execution: it is not enough for state transitions to be provably correct; the *data needed to reconstruct state and contest fraud must be provably published*. Rollups are, in one sentence, Plasma with the data availability problem solved by brute force — post the data on-chain — and the entire subsequent history of the field (blobs, DAS, validiums, external DA layers) is the story of making that brute-force step cheap without reintroducing Plasma's flaw. Notably, Plasma-style constructions retain a niche for payment-like applications with bounded state, and saw renewed academic interest once validity proofs could replace fraud proofs for exits.

3.4 Rollups: the dominant paradigm

A rollup executes transactions on its own chain (L2), then posts to the base layer (L1): (a) the transaction data (or a commitment to it, with the data guaranteed available), and (b) a state commitment after execution, whose correctness is enforced either by *fraud proofs* (optimistic rollups) or *validity proofs* (ZK rollups). Because the L1 contract holds the canonical bridge and verifies state transitions, a rollup — unlike a sidechain — inherits L1 security for state correctness: no rollup operator can steal funds by posting an invalid state root, and because transaction data is on L1, anyone can reconstruct the rollup state and exit permissionlessly even if every rollup operator vanishes.

What “inheriting L1 security” actually means

The phrase *inherits the security of the base layer* is the most-used and least-examined claim in the L2 literature, so we now unpack it with the precision it deserves. It is not an incantation; it is a checkable reduction: **a property of the L2 is “inherited” when violating it requires violating a property of the L1 itself**. For a properly constructed rollup, three specific properties reduce this way:

1. **Ledger definition and ordering.** The canonical L2 chain is *defined* as the output of applying the rollup's public state-transition function to the batch data posted on L1, in L1 order. There is no other source of truth. Consequently, rewriting L2 history requires reorganising Ethereum, and “the rollup's ledger” is exactly as immutable as the L1 blocks that carry it. (A sidechain's ledger, by contrast, is defined by its own validators' signatures — a different and weaker object.)
2. **State validity.** The bridge contract on L1 accepts a state root only if it passes the proof system — a SNARK/STARK verification executed by every Ethereum validator, or a fraud-proof game any honest party can win. Stealing funds via an invalid state root therefore requires either breaking the proof system or corrupting Ethereum consensus itself.
3. **Censorship resistance and exit.** The forced-inclusion inbox and withdrawal path are L1 contracts. If Ethereum will include Alice's transaction, the rollup *must* eventually process it; her ability to exit reduces to her ability to get one transaction into an Ethereum block.

Equally important is what is **not** inherited, because every real-world disappointment with L2s lives on this list: *liveness* (a dead sequencer stalls the chain — Ethereum guarantees

you can eventually exit, not that the L2 keeps producing blocks); *soft-confirmation integrity* (the sequencer's instant receipts are its own promise, not Ethereum's); *fair ordering / MEV protection* (inherited from nobody); *proof-system implementation soundness* (a circuit bug is the rollup's bug, not Ethereum's — Section 9.3); and *governance* (an upgrade key can replace all of the above — the Stage framework of Section 3.6 measures exactly how much of the “inheritance” is revocable by whom).

The abandonment test, step by step. The operational meaning of inheritance is best demonstrated by the worst case. Suppose Alice holds 100 USDC on a Stage-2-style rollup, and *every* operator — sequencer, proposer, prover, the company, its website — vanishes tonight, or worse, turns actively hostile. The claim “Alice's funds are secured by Ethereum” means the following procedure works, using nothing but an Ethereum node and the rollup's open-source software:

1. *Recover the data.* Alice (or anyone — one honest party suffices for all users) downloads the rollup's batch history: blob data from L1 within the ~18-day retention window, or from the archival mirrors whose correctness is checkable against the KZG commitments that remain on L1 forever. No operator's cooperation is involved; the data's availability was guaranteed by Ethereum consensus at posting time.
2. *Recompute the state.* She replays the batches through the rollup's public state-transition function from genesis. This yields the full L2 state — every account, every balance — including her 100 USDC. The vanished operator held no secret needed for this step; that is the entire point of posting data on-chain, and the step that fails on every other architecture.
3. *Prove her balance.* She computes a Merkle proof that her account leaf is included in the last *finalised* state root recorded in the L1 contract.
4. *Neutralise a hostile proposer, if any.* If the operator's final act was posting a *false* root (crediting itself Alice's funds), that root cannot finalise: on a ZK rollup it was never postable without a valid proof; on an optimistic rollup, Alice — or any of the thousands of parties able to run step 2 — submits the fraud proof during the window, and the L1 contract, not any committee, adjudicates by executing the disputed step itself.
5. *Force the exit.* She submits a withdrawal against the finalised root through the L1 escape-hatch path (forced inclusion, or the frozen-mode direct withdrawal of StarkEx-style systems). No sequencer signature is needed; delay is bounded by contract constants.
6. *Collect.* The bridge contract verifies her Merkle proof against the root it holds and releases the 100 USDC. Every verification in the chain of custody — data inclusion, root validity, proof of balance — was performed by Ethereum validators executing public code.

Now run the same drill on a sidechain with the same TVL: step 1 fails (the transaction data lives on the sidechain's own nodes, which are gone or lying), so step 2 is impossible, so there is nothing to prove in step 3; the bridge in step 6 answers to the sidechain validators' multisig, which is gone or hostile. Alice's funds are lost — not because any cryptography failed, but because the *reduction to L1 was never constructed*. This is the entire practical content of the phrase: security inheritance is not a property of intentions or token branding

but of the six steps above having code paths on L1 that work when every operator is adversarial. It is exactly what the exchange collapses of 2022 (FTX above all) demonstrated by counterexample at the custodial extreme, what Plasma could not deliver for general computation (step 1 fails under data withholding — Section 3.3), what a validium delivers only conditionally (steps 1–2 depend on the DAC honouring data requests — Section 4.4), and what L2Beat’s stages measure the *revocability* of (an upgrade key that can swap the bridge contract can delete steps 4–6 faster than users can react — Section 9.4). Section 10’s fifth code demonstration executes this drill end to end: it abandons a simulated rollup and a simulated sidechain mid-flight and shows the exit succeeding from L1 data alone in the first case and failing in the second.

The scaling win is compression and amortisation: one L1 blob carries thousands of L2 transactions; one validity proof (a few hundred kilobytes, verified in milliseconds) attests to millions of computational steps. As of mid-2026, L2Beat — the de facto independent scoreboard of the ecosystem — tracks on the order of 70+ rollups securing roughly \$45–50 billion, with Arbitrum One (~\$17 billion), Base (~\$11 billion) and OP Mainnet leading, optimistic rollups holding roughly 80% of L2 value and ZK rollups the remainder, and aggregate L2 throughput running an order of magnitude above L1’s.

Life of a rollup transaction. Before dissecting the proof paradigms, it is worth walking one transaction through the full machine, because every security property discussed later attaches to a specific step of this pipeline.

1. *Submission.* Alice signs an L2 transaction and sends it to the rollup’s sequencer (or, if she suspects censorship, directly to the L1 inbox contract — the forced-inclusion path of Section 6).
2. *Soft confirmation.* The sequencer orders and executes the transaction against its local copy of L2 state and returns a receipt within milliseconds. This is a *promise*, backed at this instant by nothing but the sequencer’s reputation (and, in preconfirmation designs, a slashable bond).
3. *Batch publication.* Within seconds to minutes, the sequencer compresses the transaction into a batch and posts it in a blob on L1. From this moment the transaction’s *ordering and content* are as final as Ethereum itself: any honest party can download the blobs and reconstruct the entire L2 state independently. Most of the trust has now drained out of the system — data availability is what made that possible.
4. *State commitment.* A proposer posts the post-batch state root to the rollup’s L1 contract — accompanied by a validity proof (ZK rollup), or bonded and presumed valid pending the fraud window (optimistic rollup). Only now do *cross-domain* consumers (the canonical bridge above all) trust the result.
5. *Settlement/withdrawal.* If Alice’s transaction was a withdrawal, the bridge releases her funds on L1 after step 4 completes — minutes to hours for ZK, seven days for optimistic (unless a liquidity provider fronts it for a fee, taking the finalisation risk themselves).

The pipeline explains the recurring exam question “when is a rollup transaction final?”: it has *three* answers — sequencer promise (ms), data finality (L1 inclusion of the batch,

~minutes), and state finality (proof verified or window expired) — and each answer protects a different party. Users trading within the L2 rely on the first; the L2's own state integrity rests on the second; the bridge rests on the third.

We now examine the two proof paradigms in depth, then the data availability layer beneath them, then sequencing.

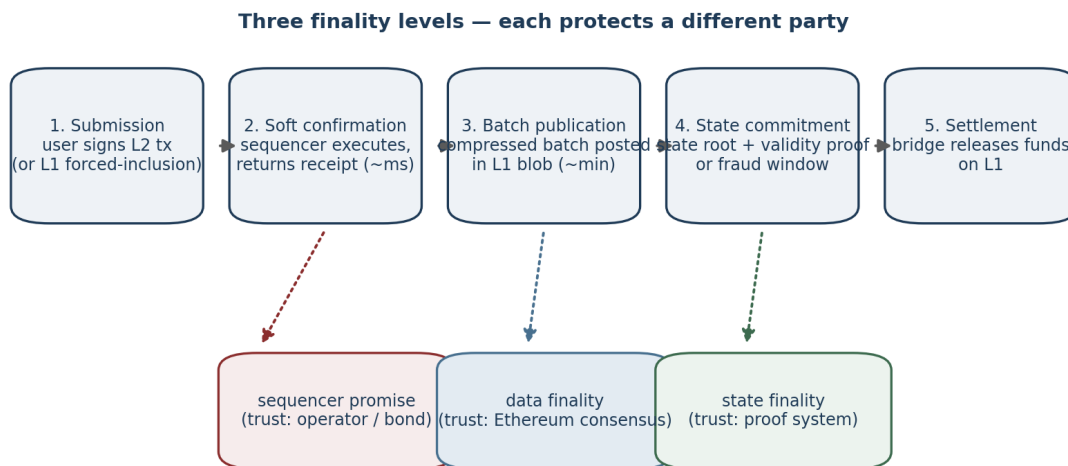


Figure 3.1 — Rollup transaction lifecycle and the three finality levels.

3.4.1 Optimistic rollups and fraud proofs

An optimistic rollup posts state roots *without proof*, presumed valid unless challenged during a dispute window (conventionally seven days). Anyone holding the posted transaction data can re-execute the batch; if the posted root is wrong, they submit a fraud proof and the contract reverts the state root and typically slashes the proposer's bond.

The engineering subtlety is *how* the L1 adjudicates fraud without re-executing an entire batch (which could exceed L1 gas limits). The solution is the **interactive bisection game**, introduced by Arbitrum (Kalodner et al., USENIX Security 2018): challenger and defender repeatedly bisect the disputed execution trace until they disagree about a *single instruction*, which the L1 contract then executes natively — a one-step proof. Arbitrum compiles its node to WASM/WAVM for this purpose; OP Stack's fault-proof system compiles a Go-based EVM (via Cannon) to MIPS and disputes over that trace.

Fraud-proof systems must also survive *adversarial economics*, not just adversarial computation:

- **Delay attacks.** In older designs, a well-funded attacker could post many bonded claims and force sequential disputes, delaying finality far beyond seven days. Arbitrum's BoLD protocol (deployed 2025) restructures disputes as an all-vs-all bonded tournament resolvable in bounded time regardless of the number of adversarial claims, enabling *permissionless* validation — previously Arbitrum restricted challengers to a whitelist precisely because of this attack.

- **Verifier’s dilemma.** If fraud never happens, verifiers earn nothing and stop checking; if nobody checks, fraud pays. Production systems answer with large proposer bonds, altruistic/institutional verifiers, and increasingly with the plan of adding validity proofs so that the game is never played at all.
- **Censorship of the challenge.** A fraud proof is useless if it cannot be included on L1 within the window. This is why the window is *days* long: it must exceed any plausible L1 censorship/reorg horizon. The seven-day exit delay — and the centralised “fast withdrawal” liquidity market it spawned — remains optimistic rollups’ principal UX tax.

By 2026, the leading optimistic systems (Arbitrum One, OP Mainnet, Base) all run *permissionless* fraud proofs in production and have reached “Stage 1” security (Section 3.6); the OP Stack’s Superchain additionally shares one upgrade path and security council across many chains (Base, OP Mainnet, Ink, and dozens more), turning rollup deployment into a commodity.

3.4.2 ZK rollups and validity proofs

A ZK rollup posts, with each state root, a succinct cryptographic proof (SNARK or STARK) that the root is the correct result of executing the posted batch from the previous root. The L1 verifier contract checks the proof; invalid states are *unpostable* rather than merely challengeable. Consequences: no dispute window (withdrawals in minutes to hours, bounded by proof cadence rather than by seven days), no verifier’s dilemma, and a smaller honest-party assumption — safety reduces to the soundness of a proof system and the correctness of a verifier contract.

The term “ZK” is a historical misnomer in this context: rollup validity proofs use succinctness, not zero-knowledge secrecy (the data is public). The genuinely hard problem was never proving *a* computation but proving *the EVM* — an architecture aggressively hostile to arithmetisation (256-bit words, Keccak hashing, RLP encoding, gas semantics). Buterin’s zkEVM taxonomy classifies solutions by fidelity:

- **Type 1** (fully Ethereum-equivalent, proves actual Ethereum blocks): the target of the Ethereum Foundation’s L1-zkEVM programme; Taiko is the closest production instance.
- **Type 2 / 2.5** (EVM-equivalent, minor gas/precompile deviations): Scroll, Linea, Polygon zkEVM.
- **Type 3** (almost-EVM, developer porting needed): earlier versions of the above.
- **Type 4** (high-level-language compatible, custom VM): ZKsync Era (compiles Solidity to its own VM), Starknet (Cairo VM — a from-scratch STARK-native architecture).

The zkVM revolution. The decisive shift of 2024–2026 was from hand-built zkEVM circuits to *general-purpose zkVMs*: prove a standard instruction set (overwhelmingly RISC-V), then run an ordinary EVM client (Geth, Reth) *inside* the zkVM. RISC Zero pioneered the pattern; Succinct’s SP1, Brevis’s Pico, a16z’s Jolt (notable for design simplicity and auditability), ZKsync’s Airbender, Axiom’s OpenVM and others industrialised it. This collapsed the engineering problem — any code a compiler can target becomes provable —

and turned proving performance into a benchmarkable commodity. Performance followed a near-Moore trajectory: Ethereum-mainnet-block proving fell from tens of minutes (2023) to ~16 seconds and then into *real time* (under the 12-second slot) during 2025–2026, with SP1’s 2026 releases proving mainnet blocks several times faster than real time on GPU clusters. Proving cost per transaction fell correspondingly, from dominant to marginal (fractions of a cent for simple transfers), though proving remains a genuine industrial operation — GPU fleets, specialised proving networks and markets (Succinct Network, Boundless, Fermah), with dedicated ASICs in development.

Real-time proving unlocks the two most consequential items on Ethereum’s current roadmap:

- **The L1 zkEVM / lean Ethereum plan** (Ethereum Foundation, 2025–2026): validators verify a succinct proof of each L1 block instead of re-executing it, with a multi-prover regime (several independent zkVMs proving each block, so one prover bug cannot forge state) and stringent published targets for provers (real-time latency at ≤ 10 kW power for the home-prover tier, 99%+ of blocks) so that proving itself does not centralise. This makes gas limits of 100M–1G realistic without raising verifier costs — the trilemma escape applied to L1 itself, targeting on the order of 10,000 TPS on L1 per the Lean Ethereum roadmap laid out in July 2026.
- **Native rollups** (EIP-9698 proposal, 2026): an EXECUTE precompile letting L2s delegate EVM state-transition verification to the L1 itself, making L2s “programmable extensions” of Ethereum that inherit L1 execution semantics and upgrade in lockstep — eliminating the divergence and upgrade-key risk that plagues today’s rollups (Section 9.4).

Two caveats belong in any honest account. First, circuit/prover *soundness bugs* replace fraud-window risk: a bug in a hand-written circuit or a zkVM compiler is a direct path to forged state, and several such bugs (fortunately caught before exploitation) have been disclosed in production systems — hence multi-provers, audits, formal verification of verifier contracts, and (interim) training-wheel mechanisms like security councils able to override a provably-wrong root. Second, ZK rollups in production today still mostly run *centralised provers and sequencers*; the proof guarantees state validity, not liveness or fair ordering.

3.4.3 Hybrid designs

The paradigms are converging. Optimistic stacks are adding validity proofs (OP Stack’s roadmap allows ZK fault proofs via zkVMs proving the dispute program; Arbitrum has prototyped BoLD-with-ZK one-step proofs), yielding *hybrid* systems where a validity proof can finalise instantly but a fraud-proof game remains as a fallback — a defence-in-depth pattern (any one honest mechanism suffices) that many researchers regard as the correct end state for high-value systems.

3.5 Rollup economics: what users actually pay

A rollup transaction’s cost decomposes into: (a) L2 execution (cheap — one operator’s marginal compute), (b) L1 data cost (the dominant term historically), and (c) amortised

proving/settlement overhead (state-root posting, proof verification — fixed per batch, negligible per transaction at scale). The history of rollup fees is therefore mostly the history of L1 data pricing:

- **Calldata era (2019–March 2024).** Rollups posted compressed transaction data as L1 calldata at 16 gas/byte, competing with all other L1 gas users. Typical L2 fees: \$0.10–\$0.50 for transfers, dollars in congestion.
- **EIP-4844 blobs (Dencun, March 2024).** Proto-danksharding introduced *blobs*: ~128 KB data packages carried by beacon blocks, priced by an independent EIP-1559-style fee market, pruned after ~18 days (data availability, not permanent storage, is the requirement — long enough to cover any fraud-proof window with margin). With demand initially below the 3-blob target, blob fees fell to near zero and median L2 fees dropped to fractions of a cent — routinely a 10–100× reduction. Pectra (May 2025) raised target/max to 6/9 blobs.
- **PeerDAS era (Fusaka, December 2025 onward).** Data availability sampling (Section 4.3) removed the requirement that every node download every blob, allowing blob counts to rise via scheduled *blob-parameter-only (BPO) forks* — pre-programmed increases (target 10/max 15 shortly after activation, stepping toward a 48-blob maximum) gated on observed network health, with further increases queued behind Glamsterdam. Blob supply now scales ahead of demand as a matter of explicit protocol policy.

A worked decomposition makes the arithmetic concrete. Consider an ERC-20 transfer on a major optimistic rollup in the blob era. Its compressed on-chain footprint is on the order of 15–25 bytes (nonce and amounts delta-encoded, signatures aggregated or replaced by validity conditions, addresses dictionary-compressed across the batch). At a blob base fee near the protocol minimum, the DA cost of those bytes is a few thousandths of a cent; at the congested peaks of 2024–2025 blob-fee spikes it briefly reached whole cents. To this add the transaction’s share of the batch’s fixed costs: posting the state root and, for a ZK rollup, verifying the proof — roughly 300k–500k gas per batch, which at 10,000 transactions per batch contributes 30–50 gas-equivalents each, i.e., another small fraction of a cent. L2 execution adds a marginal operator cost that is smaller still. The sum — well under a cent — explains the post-Dencun fee tables; the *structure* of the sum explains two design pressures: batches want to be large (fixed costs amortise) and data wants to be small (the marginal term dominates), which is why compression engineering and proof aggregation, unglamorous as they are, have been among the highest-leverage work in the ecosystem.

Two economic subtleties deserve attention. First, *fee asymmetry risk*: when blob prices spike (as during 2024–2025 inscription and airdrop waves), rollups that quote users fees before buying blob space bear basis risk, and several were briefly posting at a loss; production systems now use predictive fee models with margins. Second, *sequencer revenue*: the spread between L2 fees collected and L1 costs paid — plus MEV — accrues to the sequencer operator, which is why sequencing rights (Section 6) are both an economic prize and a governance question. Base alone generates on the order of \$100 million annualised sequencer revenue; “who captures this, under what obligations” is a live policy

debate inside every rollup ecosystem, and the substance behind “based rollup” designs that redirect sequencing to L1 validators.

3.6 How secure is a given L2? The stages framework

Because “rollup” describes an architecture, not a guarantee, the community (led by L2Beat) formalised a maturity ladder that has become the industry’s shared vocabulary:

- **Stage 0** — “full training wheels”: the operator can post state without functioning proofs, or a committee can override anything; users rely on the operator’s honesty. (Examples in 2026: ZKsync Era and Linea remain Stage 0 pending permissionless proof mechanisms.)
- **Stage 1** — proofs run permissionlessly and users can exit without operator cooperation, but a security council (typically an 8-of-12-style multisig meeting strict independence criteria) can still upgrade/override with limited delay. (Arbitrum One, OP Mainnet, Base, Starknet, Scroll and others reached Stage 1 across 2024–2026 — the year Base and OP Mainnet shipped permissionless fault proofs was the ecosystem’s most important security milestone to date.)
- **Stage 2** — the endgame: upgrades delayed longer than the exit window (≥ 30 days) so users can always leave before rules change; council powers restricted to provable-bug states. Only a handful of systems (historically DeGate, Fuel) have claimed it; the ecosystem’s centre of mass is Stage 1.

The framework’s deeper lesson generalises beyond rollups: *every* scaling system should be interrogated with the same three questions. Who can post state, and what stops an invalid state? Who can censor, and what forces inclusion? Who can change the rules, and how fast? We will apply exactly these questions to attacks in Section 9.

4. The data availability problem

Data availability (DA) is the load-bearing concept of modern scaling, and the reason this chapter has already mentioned it a dozen times. We now treat it properly.

4.1 The problem

Fraud proofs and permissionless exits share a prerequisite: challengers and users must be able to *obtain the data* behind each posted commitment. A malicious operator who publishes a state root but withholds the underlying transactions has not (yet) committed provable fraud — the root might be valid — but has disabled everyone else’s ability to check, exit, or continue the chain. Plasma died on this rock (Section 3.3). Crucially, data *withholding* is not attributable: if Alice claims “block data is missing” and Bob claims he published it, a third party cannot adjudicate without fetching the data — and unavailable-then-released data makes the accuser look dishonest. This is why DA cannot be solved by fraud proofs and needs its own machinery.

The naive solution — every verifier downloads everything — is exactly the $O(T)$ bandwidth wall of Section 1.3. The modern solution stack rests on two primitives: erasure coding and polynomial commitments, combined into *data availability sampling*.

4.2 Erasure coding and KZG commitments

Reed–Solomon extension. Encode the block data of n chunks into $2n$ chunks such that *any* n of the $2n$ suffice to reconstruct the whole. Now an adversary who wants to withhold even one byte of the original data must withhold *more than half* of the extended chunks — turning “hide a needle” into “hide half the haystack,” which random sampling can detect.

KZG polynomial commitments (Kate–Zaverucha–Goldberg, ASIACRYPT 2010) bind the encoding. Data is interpreted as evaluations of a polynomial; the commitment is a single 48-byte group element; each chunk ships with a constant-size opening proof that it lies on the committed polynomial. This gives DAS its two crucial properties: samples are *verifiable* (a fake chunk is rejected immediately) and the encoding is *provably well-formed* (KZG commits to a degree-bounded polynomial, so incorrectly extended data is impossible rather than merely challengeable — eliminating the fraud-proof-for-bad-encoding round trip that fraud-proof-based DA schemes like Celestia’s must retain). The cost: KZG requires a trusted setup (Ethereum’s KZG ceremony, 2023, with $\sim 140,000$ participants — secure if *any one* participant was honest) and pairing-based cryptography that is not post-quantum, which is why the Lean Ethereum track plans an eventual migration to hash-based commitments.

For readers tracking the mathematics: interpret each blob as evaluations of a polynomial p of degree $< n$ over a chosen domain. The KZG commitment is $C = [p(\tau)]_1$ — p evaluated at a secret point τ *inside the exponent* of an elliptic-curve group, computable from the ceremony’s powers-of- τ without anyone knowing τ . To open the commitment at point z , the prover supplies $q = (p - p(z))/(X - z)$ committed the same way, and the verifier checks one pairing equation asserting $p(X) - p(z) = (X - z) \cdot q(X)$ at τ . Soundness rests on the fact that a cheating prover would need to know τ to fabricate a consistent quotient. Three consequences matter for scaling: openings are constant-size (one group element) regardless of blob size; commitments are *additively homomorphic*, which is what allows the Reed–Solomon extension and row/column structure of danksharding to be verified without re-encoding; and the degree bound is enforced by the setup itself, which is why a KZG-committed extension cannot be malformed — the property that lets validity-style DA (Ethereum, Avail) drop the encoding-fraud-proof machinery that fraud-proof-based DA must carry.

4.3 Data availability sampling and PeerDAS

The idea of DAS (formalised in Al-Bassam, Sonnino & Buterin’s 2018–2019 fraud/DA proofs work and the later DankSharding designs): each light verifier requests k random chunks of the extended data. If the data is more than half withheld (the minimum to actually hide anything), each query independently fails with probability $\geq \frac{1}{2}$, so k successful samples bound the probability of undetected withholding by $2^{(-k)}$ — thirty samples of a few kilobytes give $\sim 10^{-9}$ assurance about megabytes of data. Collectively, a

few thousand samplers reconstruct everything sampled with overwhelming probability, so data that passes sampling is not merely “probably published” but *recoverable by the honest network*.

PeerDAS in production (Fusaka, December 2025). Ethereum’s first deployed DAS implements this at the peer-to-peer layer: each blob’s extended data is arranged into a matrix and split into 128 *columns*; every node subscribes to at least 8 randomly assigned column subnets (custody), receiving 1/16 of the extended data (= 1/8 of original); validators custody more, proportional to stake; full nodes can reconstruct any blob from any 50% of columns. The immediate effect was to change node bandwidth from $O(\text{blobs})$ to $O(\text{blobs}/16)$, unlocking the BPO-fork scaling schedule of Section 3.5 — the concrete mechanism by which “Ethereum as DA layer” scales through 2026–2027 toward full Danksharding (2D erasure coding, sampling in both rows and columns, and eventually cell-level sampling so that even resource-limited nodes sample only kilobytes).

An important operational nuance: DAS provides its full guarantee to a node only if that node’s samples are unlinkable and its network vantage honest — an eclipse-attacked sampler can be shown exactly the chunks it asks for while the rest are withheld. Production designs mitigate with diverse peer selection, subnet randomisation, and reconstruction gossip, but “DAS under network-level adversaries” remains an active research area (Section 13).

4.4 The DA marketplace: validiums, DACs, and external DA layers

Once execution (rollup) and data (DA) are separated, data can in principle live anywhere — giving rise to a spectrum of designs distinguished, as always, by trust assumptions:

- **Rollup** (data on L1): full L1 DA security. The only mode L2Beat counts as a true rollup.
- **Validium** (validity proofs + off-chain data): state correctness is cryptographically guaranteed, but a data-withholding operator/committee can freeze users’ ability to exit (they cannot prove account inclusion without data). StarkEx validiums (dYdX v3 historically, Immutable X) pioneered the model, with a *Data Availability Committee* (DAC) — n known entities signing that they hold the data, secure if a threshold remains honest and online.
- **Optimium/optimistic chain** (fraud proofs + off-chain data): the weakest combination — withheld data disables the fraud proofs themselves (the Plasma problem returns). Treated by analysts as sidechain-adjacent.
- **External DA layers**: dedicated chains selling verifiable data availability, competing on the cost/security frontier between “DAC” and “Ethereum blobs”:
 - **Celestia** (mainnet 2023; Mustafa Al-Bassam co-founder — the LazyLedger lineage): a minimal chain doing *only* ordering + DA, with namespaced Merkle trees for per-rollup data retrieval and light nodes performing DAS against 2D Reed–Solomon-coded blocks; fraud proofs guard encoding correctness. 8 MB blocks in production, ~128 MB in the 2026 Matcha upgrade, with the research roadmap (1 GB blocks) explicitly targeting orders of magnitude beyond Ethereum’s blob capacity; roughly half the external-DA market by data posted.

- **EigenDA** (2024, on EigenLayer): restaked-ETH operators attest to data custody — architecturally a scaled DAC with cryptoeconomic slashing rather than a sampled public network; v2 targets 100 MB/s. Highest throughput, heaviest trust assumptions among the majors.
- **Avail** (2024, Polygon spin-out): KZG-committed DAS chain — validity-proved encoding (no fraud-proof window on encoding), 4 MB blocks scaling via a roadmap toward multi-GB.

The pattern to internalise: *DA choice is the single biggest security decision an L2 architect makes*, and it prices directly — Ethereum blobs post-Fusaka cost fractions of a cent per transaction of data; external DA costs 10–1000× less still; DACs are nearly free — while the failure mode ranges from “inherits Ethereum consensus security” to “n-of-m companies could freeze your funds.” High-value DeFi has consistently chosen rollup mode; games, social, and high-frequency perp DEXs routinely choose validium/external-DA modes. There is no correct universal answer; there is only an informed trade.

4.5 A worked decision: choosing DA for a concrete workload

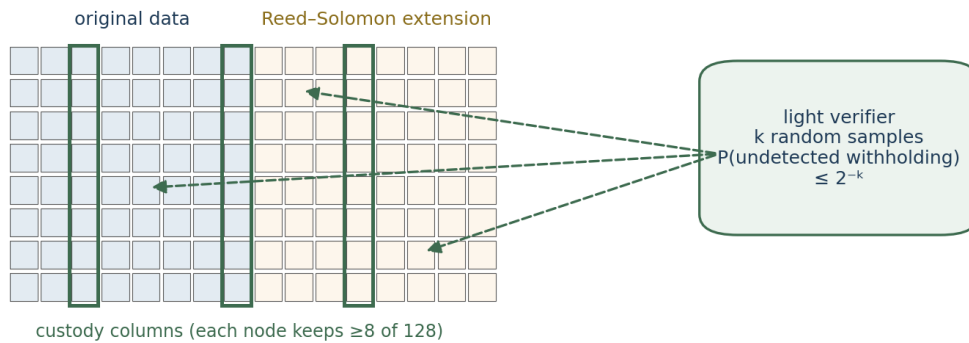
To make the trade concrete, consider two archetypes an architect actually faces.

Workload A: a lending market expecting \$2B TVL, ~20 TPS. Data volume is trivial (a few KB/s), so DA cost is irrelevant even at blob prices; what matters is that a data-withholding event against \$2B of positions would be catastrophic and headline-making. Choose Ethereum blobs (rollup mode): the DA cost of the entire protocol is dollars per day, and the security statement to users is one sentence long. Any other choice trades an invisible cost saving for a visible new trust assumption.

Workload B: a perp DEX targeting 5,000 TPS of order flow. Order-book updates generate MB/s of data; on blobs this is materially expensive and contends with every other rollup for BPO-scheduled supply; on a DAC it is free but the exchange’s own committee could freeze the book. The productionised answers: validium with a reputable DAC plus StarkEx-style freeze-and-exit (dYdX v3’s historical choice — users frozen-not-robbed under withholding), or an external DAS layer (Celestia/Avail) that restores public verifiability at ~100× below blob cost, accepting a second consensus system’s token economics and light-client bridge into the security story. Hyperliquid’s answer — become an app-specific L1 and own the whole stack — is the third corner of the same triangle.

The disciplined form of the decision is always the same three questions of Section 3.6 asked against the *data*: who can withhold it, what happens to my users when they do, and what does each avoided trust assumption cost per transaction?

Data availability sampling (PeerDAS): erasure-coded matrix, column custody, random samples



withholding any original byte requires hiding >50% of extended chunks — each sample then fails with $p \geq \frac{1}{2}$

Figure 4.1 — DAS — erasure-coded matrix, column custody, and random sampling.

5. Zero-knowledge proof systems for scalability

Because validity proofs now anchor everything from ZK rollups to the L1-zkEVM plan, a scalability engineer needs a working — not merely nominal — understanding of the proof systems themselves. This section provides it, at the level of trade-offs and system design rather than full cryptographic construction (covered in this book’s cryptography chapters).

5.1 What a succinct proof is, and is not

A *succinct non-interactive argument of knowledge* (SNARK) for a computation C lets a prover convince a verifier that “there exists a witness w such that $C(x, w) = \text{accept}$ ” with a proof that is (a) short — polylogarithmic or constant in the size of C — and (b) fast to verify — much faster than running C . *Zero-knowledge* — the proof reveals nothing about w — is an optional extra property, essential for privacy applications but incidental for rollups, where the witness (the transactions) is public anyway; the industry’s “ZK rollup” terminology entrenched before this distinction mattered. What scalability consumes is **succinctness**: verification cost decoupled from computation size. That single property converts “every node re-executes everything” into “one machine executes; everyone verifies in milliseconds,” and it is the deepest of the trilemma escapes.

The workflow every proving stack shares: (1) **arithmetisation** — compile the computation into an algebraic form (R1CS constraint systems for Groth16-lineage SNARKs; AIR execution tables for STARKs; Plonkish gate/lookup tables in between); (2) **commitment** — the prover commits to its execution trace polynomials; (3) **interactive oracle proof** — prover and verifier play a randomised protocol reducing “the trace satisfies all constraints” to a few polynomial evaluation checks; (4) **Fiat-Shamir** — the interaction is made non-

interactive by deriving the verifier’s challenges from a hash transcript. Every practical difference between systems — proof size, prover speed, trusted setup, quantum resistance — traces back to choices in steps (1) and (2).

5.2 The design space: SNARKs, STARKs, and the modern synthesis

Pairing-based SNARKs (Groth16, 2016) sit at one extreme: proofs of ~200 bytes, verification in a few milliseconds via elliptic-curve pairings — but a *trusted setup per circuit* (the “toxic waste” problem: whoever knows the setup randomness can forge proofs) and no quantum resistance. Groth16 powered the first generation (Zcash, early ZK rollup verifiers) and still often terminates proving pipelines because its proofs are the cheapest to verify in the EVM.

Universal-setup SNARKs (PLONK 2019, Marlin, and the Plonkish ecosystem — Halo2, HyperPlonk) replaced per-circuit ceremonies with a single updatable universal setup, and added *custom gates* and *lookup arguments* (Plookup lineage) that make hash functions and range checks — the EVM’s bread and butter — orders of magnitude cheaper to arithmetise. Most production zkEVM circuits of the 2022–2024 era were Plonkish.

STARKs (Ben-Sasson et al., 2018) sit at the other extreme: no trusted setup, hash-based (plausibly post-quantum), scalable proving — at the price of proofs measured in tens to hundreds of kilobytes. The FRI low-degree test underlying STARKs replaced pairings with Merkle trees and hashing, which is why STARK-family systems dominate wherever prover performance and setup-freedom matter more than on-chain proof size. StarkWare’s Cairo/Stone/Stwo line and virtually all modern zkVM provers (SP1, RISC Zero’s, Plonky3-based stacks) are STARKs or STARK-SNARK hybrids.

The synthesis, and why proving got 1000× faster. Three lines of research converged during 2023–2026 to produce the real-time proving reported in Section 3.4.2. First, *small fields*: early systems worked over 256-bit fields to match Ethereum’s curves, wasting most of every multiplication; Plonky2’s 64-bit Goldilocks field, then 31-bit fields (BabyBear, Mersenne-31 in Plonky3/Stwo/circle-STARKs), and Binius’s binary-field tower let provers work in machine-word-sized arithmetic, with a final wrapping proof converting to an EVM-verifiable format. Second, *lookups and precompiles*: instead of arithmetising SHA-256 gate by gate, zkVMs prove it in dedicated accelerated tables (“precompiles”), reserving the general VM for control flow. Third, *folding and recursion*: recursion (a proof verifying other proofs) enables both unbounded computation via proof chaining and massive parallelism — split a block into thousands of chunks, prove them simultaneously on a GPU cluster, and aggregate up a tree; folding schemes (Nova, ProtoStar lineage) make the accumulation step itself cheap. GPU implementations of all of the above, plus emerging proving ASICs, complete the picture. The consequence is the benchmark trajectory already cited: Ethereum block proofs from ~minutes (2023) to seconds (2025) to faster-than-real-time (2026), and proving costs per transaction falling from dominant to fractions of a cent.

Aggregation as infrastructure. Because L1 verification carries a fixed gas cost (hundreds of thousands of gas), the economics push toward *shared verification*: aggregation layers (Polygon AggLayer’s aggregated proofs, Starknet’s SHARP shared prover, proof-aggregation markets such as Aligned and NEBRA) batch many rollups’ proofs into one on-

chain verification, amortising the fixed cost across the ecosystem — the same amortisation logic as rollups themselves, applied one level up. Proving itself has become a market: decentralised prover networks (Succinct Network, Boundless, Fermah) auction proving jobs to competing GPU operators, turning proof generation into a commodity with a spot price.

5.3 zkVMs versus zkEVMs: the industrialisation of proving

Section 3.4.2 introduced the shift from hand-built zkEVM circuits to general-purpose RISC-V zkVMs; the systems consequences deserve elaboration. A hand-built circuit is a bespoke arithmetisation of the EVM — years of specialist work, error-prone (every opcode edge case is a potential soundness bug), and frozen against EVM upgrades. A zkVM instead proves a commodity ISA and runs an *unmodified, battle-tested* execution client (Reth, Geth) inside it. The trade: perhaps 10–100× more cycles to prove (interpreting the EVM in RISC-V rather than proving it natively), which the small-field/precompile/recursion advances absorbed. What is bought is priceless at system level: **auditability** (the trusted computing base shrinks to one ISA circuit plus a compiler, rather than a full EVM re-implementation), **agility** (an EVM hard fork is a client recompile, not a circuit rewrite), and **diversity** (eight-plus independent zkVMs — SP1, RISC Zero, Jolt, Pico, OpenVM, Airbender, ZisK, LambdaVM — can prove the *same* blocks, enabling the multi-prover regimes of Section 9.3). This is why Ethereum’s L1-zkEVM programme standardises on RISC-V targets and treats zkVM diversity as a first-class security requirement, and why Buterin’s 2025 proposal to eventually *replace the EVM’s outward-facing role with RISC-V* — letting contracts target the ISA the provers are optimised for — is taken seriously as a long-horizon option.

5.4 Limits, costs, and honest caveats

Four things the marketing omits. **Proving is physical**: real-time mainnet proving in 2026 requires clusters on the order of a hundred GPUs (falling fast, with published EF targets of ≤ 10 kW — roughly one household — for the home-prover tier); until commodity hardware suffices, proving liveness depends on an industrial supply chain. **The verifier is a single point of failure**: all succinctness routes through one on-chain contract; its correctness (and the correctness of the circuit it encodes) bounds everything, motivating the formal-verification investment described in Section 9.3. **Fiat-Shamir and setup are load-bearing**: the transformation from interactive to non-interactive proof is where several published attacks on deployed systems have landed (the “Frozen Heart” vulnerability class, 2022, affected multiple implementations), and pairing-based components inherit trusted-setup and quantum exposure — the reason Lean Ethereum’s cryptography track is explicitly hash-based. **Proof latency is not zero**: even real-time proving adds seconds; ZK rollup “instant finality” claims refer to L1-verified finality being *faster than fraud windows*, not instantaneous — soft confirmations still come from a sequencer’s promise (Section 6).

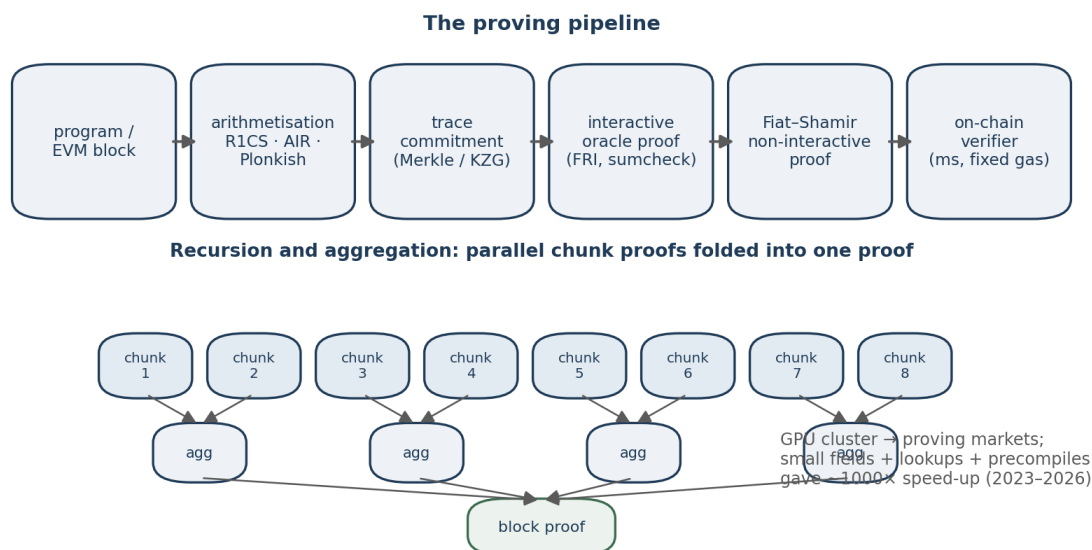


Figure 5.1 — Proving stack — arithmetisation, commitment, recursion/aggregation pipeline.

6. Sequencing: ordering, liveness, and MEV

A rollup's *sequencer* receives user transactions, orders them, executes them for instant soft confirmation, and posts batches to L1. In 2026, virtually every major rollup still runs a *single centralised sequencer* operated by its core team — the ecosystem's most-criticised deviation from its own ideals, and a recurring source of incidents: Arbitrum's sequencer halted for ~78 minutes in December 2023 under inscription-mint load and again briefly in 2024 (hardware failure); Base halted block production for 33 minutes in August 2025 (failover misprovisioning) and suffered a roughly two-hour outage in June 2026 when an invalid block froze its conductor cluster. No funds were lost in any of these — state validity was never at issue — but on-chain DeFi cannot liquidate, trade, or top up collateral while its chain is dark, which converts liveness incidents into financial risk (Section 11).

The mitigation hierarchy. Every serious rollup provides a *forced-inclusion* path: users may submit transactions directly to the L1 inbox contract, which the sequencer must include within a bounded delay (typically ~12–24 hours). This bounds censorship — the sequencer can delay but not deny — and, combined with permissionless batch posting after operator inactivity, means a dead sequencer degrades UX rather than custody. The three questions of Section 3.6 apply: soft confirmations are a *promise*, not a guarantee, until L1 data inclusion.

Decentralised sequencing replaces the single operator with a validator set running BFT consensus over ordering (Metis was an early mover; several stacks have testnets), at the cost of latency and engineering complexity — and it solves self-inflicted MEV governance as much as liveness. The design space is worth spelling out, because “decentralise the sequencer” names at least four distinct architectures with different guarantees: (a) *leader rotation* over a permissioned set — the simplest, converting single-operator failure into

single-epoch failure, with the set itself still a governance choice; (b) *BFT consensus sequencing* — a Tendermint/HotStuff-family quorum orders batches, giving real fault tolerance at 1–2 s of added latency and reintroducing every validator-set question (selection, stake, slashing, cartelisation) one layer up; (c) *auction-based sequencing* — sequencing rights for a time slice are sold (Arbitrum’s timeboost being the deployed example of an adjacent idea, auctioning priority *within* an ordering policy), which makes MEV explicit and redistributable but concentrates operation in auction winners; and (d) *based sequencing* — delegation to L1, discussed below. A useful discipline when evaluating any of them: ask what happens to (i) soft-confirmation latency, (ii) censorship of a specific transaction, and (iii) MEV distribution, under one crashed node, one malicious node, and a malicious third of the set. The four architectures give materially different answers, and whitepapers rarely volunteer them. **Shared sequencing** (Espresso, Astria) proposes one decentralised sequencer set serving *many* rollups, enabling atomic cross-rollup inclusion (and, with more machinery, atomic execution) — an elegant answer to fragmentation that has struggled commercially, because sequencing revenue is precisely what rollup operators are least eager to share.

Based sequencing. A *based rollup* (Justin Drake’s 2023 formulation; Taiko the flagship implementation since May 2024) has no sequencer at all: L1 proposers/searchers sequence L2 blocks as part of L1 block production. Liveness and censorship resistance become exactly Ethereum’s; sequencing MEV flows to L1 validators; cross-rollup atomicity among based rollups sharing an L1 proposer becomes natural. The historical objection — 12-second L1 slots make for terrible UX — is answered by *preconfirmations*: bonded L1 proposers (or a designated preconfirm set) sign sub-second promises to include transactions, slashable if broken; Taiko shipped sub-second preconfirmations in production during 2025–2026 with a decentralised preconfirm roadmap. Based sequencing plus native execution verification (EIP-9698) is the maximal “L2 as pure extension of L1” design, and ENS’s Namechain (Taiko-stack, 2026 testnet) signals meaningful adoption. Its economics remain the open question: an L2 that gives away sequencing revenue funds itself how?

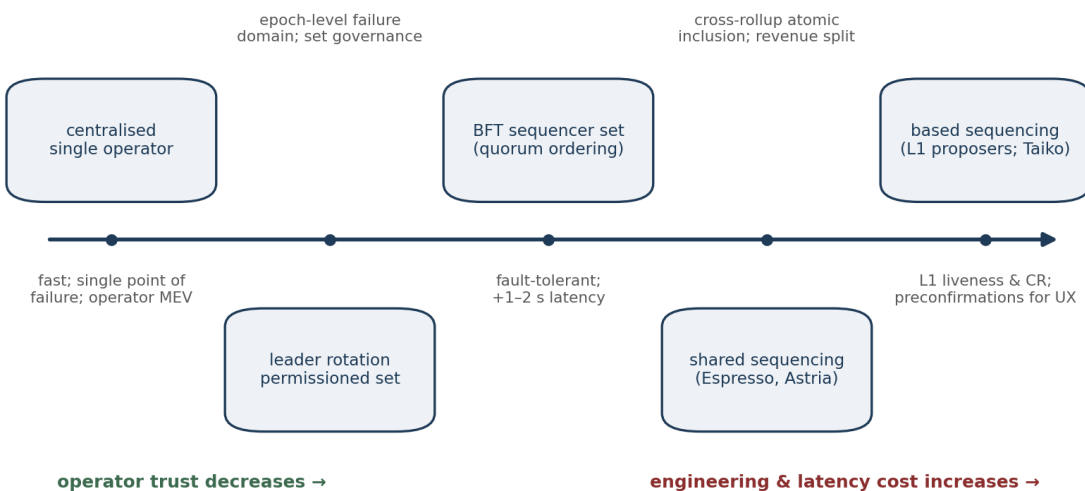


Figure 6.1 — Sequencing spectrum — centralised, decentralised, shared, based.

MEV in the scaled world. Maximal extractable value — profit from ordering discretion (sandwiches, liquidation races, arbitrage) — does not disappear on L2; it concentrates in whoever orders. Centralised sequencers today mostly run first-come-first-served with private mempools (no public sandwiching, but users must *trust* the operator’s ordering); decentralised designs must import the whole L1 MEV toolkit (encrypted mempools, commit-reveal, order-flow auctions like MEV-Boost/SUAVE lineage, threshold decryption as in Shutter). A quantitative note for perspective: cross-domain MEV (arbitrage between L1, many L2s and CEXs) is now a larger category than intra-chain sandwiching, and is one of the strongest economic forces pushing toward shared/based sequencing and toward the intent architectures of Section 7.

7. Interoperability: scaling’s second-order problem

Scaling by many chains created the problem scaling was meant to solve, one level up: liquidity, users, and state are now *fragmented* across dozens of rollups and hundreds of chains. A user’s assets on Arbitrum cannot natively pay a contract on Base; every hop is a trust decision. The interoperability stack has evolved through three generations.

Generation 1 — external bridges. Lock-and-mint bridges holding assets in a contract (or worse, a multisig) controlled by an external validator set. This design produced the worst security record in the industry’s history — roughly \$2.8–3 billion stolen from bridges since 2021, ~40% of all Web3 hack value (Section 9.1) — because it reintroduces exactly the trusted-committee model that rollups were designed to eliminate.

Generation 2 — light-client/proof-based bridges and rollup-native paths. Verify the source chain’s consensus or state proof on the destination (IBC in Cosmos as the gold standard among homogeneous chains; ZK light clients — e.g., Succinct/Polyhedra-style Ethereum light-client proofs, now also underpinning “ZK IBC” between heterogeneous

chains — making this affordable on Ethereum-family chains). Rollup-native withdrawals are the trust-minimal baseline (only L1 + proof system assumptions) but pay latency (an epoch to a week). Aggregation layers industrialise this: Polygon’s AggLayer (2024–2026) posts a single aggregated proof for many chains with a *pessimistic proof* ensuring no chain can withdraw more than was deposited across the whole set — unified bridging with accounting-level containment of a compromised member chain; the OP Superchain standardises native cross-chain messaging across OP-Stack chains behind a shared security council.

Generation 3 — intents. Rather than users driving step-by-step bridge mechanics, the user signs an *intent* — “I want $\geq X$ USDC on Base, here is Y USDC on Arbitrum” — and competing professional *solvers* front the destination-side outcome immediately, assuming the settlement risk and latency themselves and being repaid through the canonical (slow, trust-minimal) path. ERC-7683 (Across + Uniswap, 2024; adopted by 50+ projects) standardises the intent interface; the Ethereum Foundation’s Open Intents Framework (2025) and its Ethereum Interop Layer work aim to make cross-L2 UX feel like one chain, with solver networks settling billions monthly by 2026. Intents do not eliminate trust — they professionalise it: users get instant, competitive execution; solvers, who *can* evaluate bridge risk, carry it; and the slow trust-minimal path remains the settlement backstop and the ceiling on what solvers will quote. Chain abstraction — wallets and account-abstraction infrastructure hiding chains entirely — is the UX end-state being built on top.

Because bridge choice is a security decision of the same rank as DA choice, the trust spectrum deserves its own table:

Table 7.1 — Interoperability mechanisms by trust assumption

Mechanism	Verifies	Trust assumption	Latency	Historical failure mode
Rollup-native withdrawal	validity/fraud proof on L1	L1 + proof system only	minutes–7 days	none to date (slow, not unsafe)
Light-client / IBC	source consensus on destination	source chain’s validator set	~finality time	source-chain 51%/long-range attacks (theoretical)
ZK light-client bridge	succinct consensus proof	proof soundness + relay liveness	minutes	circuit bugs (Section 9.3 class)
Aggregation layer (AggLayer-style)	aggregated validity + pessimistic proof	aggregator + member-chain proofs; losses contained per chain	minutes	none major to date
External validator bridge	committee signatures	n-of-m signers	seconds–minutes	Ronin, Harmony, Multichain... (Section 9.1)

Liquidity network / intents (ERC-7683)	solver fronts outcome; canonical path settles	solver bonding + underlying settlement path	seconds	solver insolvency; underlying bridge risk repriced, not removed
--	---	---	---------	---

The architectural summary: the ecosystem is converging on *trust-minimised settlement + market-based fast paths*, the same layering (slow safe base, fast credit on top) that traditional finance evolved, with correspondent banking as the historical analogue of solver networks.

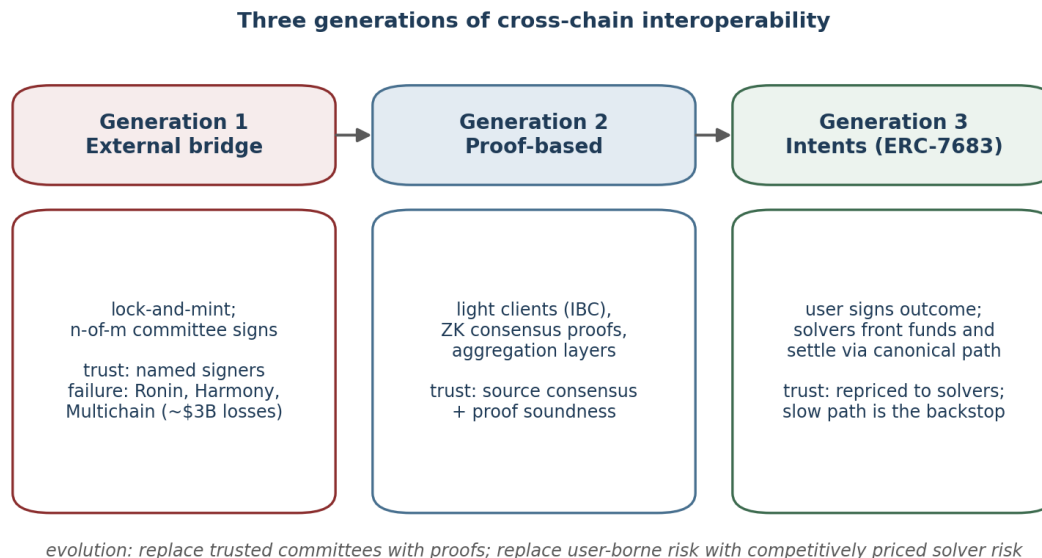


Figure 7.2 — Three generations of interoperability — external bridges, proof-based, intents.

8. The state of the art: a landscape snapshot (mid-2026)

It is useful to pause and consolidate the moving pieces into one picture of where each major ecosystem stands and where its roadmap points. (Numbers in this section are approximate, current to mid-2026, and cited to the primary sources in Further Reading; instructors should expect them to date faster than the concepts around them.)

Ethereum. The rollup-centric roadmap is functioning: L1 targets *verifiability and data*, L2s target execution. Recent and pending milestones, in order: Dencun/EIP-4844 blobs (2024) → Pectra (2025: 6/9 blobs, EIP-7702 account-abstraction improvements) → Fusaka (Dec 2025: PeerDAS, BPO forks stepping blobs toward 48) → Glamsterdam (H2 2026: ePBS/EIP-7732, BALs/EIP-7928, gas limit path 60M → ~200M) → Hegotá (FOCIL inclusion lists) → the Lean Ethereum programme (announced by Buterin and the EF in July 2026): L1 zkEVM verification with a multi-prover regime, hash-based/post-quantum cryptography, SNARKed consensus (“Beam”-lineage), targeting on the order of 10,000 TPS at L1 and ~1M TPS aggregate across L2s over a multi-fork horizon. L2s secure ~\$45–50B; the leading

optimistic stacks run permissionless fault proofs at Stage 1; ZK stacks lead on withdrawal latency and are converging on Stage 1; real-time proving is demonstrated and hardening.

Bitcoin. Deliberately static L1 (~7 TPS, 10-minute blocks); scaling lives entirely off-chain. Lightning is mature payments infrastructure (~5,000+ BTC public capacity, ~\$1B+/month volume, USDT via Taproot Assets since 2026); the VTXO generation (Ark, Spark) is live and reducing operational burden; BitVM2-style optimistic bridge/verification research is bringing fraud-proof-secured sidesystems within reach without consensus changes; and CTV/CSFS covenant activation signalling (2026–2027) would materially improve every one of these. Bitcoin’s scaling ceiling is set by its governance conservatism — a deliberate trade.

Solana. The integrated (single-state-machine) counter-thesis to modular scaling: production throughput in the thousands of TPS (with Firedancer pushing toward 10k+ real-world), fees around \$0.001–0.01, Alpenglow (late 2026) cutting finality to ~150 ms, plus ZK Compression reducing state costs for mass-market applications. Its trade-offs: high validator hardware requirements (hundreds of GB RAM, datacentre bandwidth), a history of full-network outages (none since February 2024), and verification costs that price out consumer hardware — the trilemma paid in decentralisation-of-verification, partially mitigated by forthcoming light-client and proof work.

The parallel-EVM and app-specific field. Monad (mainnet Nov 2025) delivers ~10k TPS EVM-compatible throughput via pipelined Block-STM execution and MonadDB; MegaETH (rollup) targets real-time (~ms) blocks with a heavy sequencer and stateless verification; Sui and Aptos continue to lead DAG/Block-STM innovation with sub-second finality; Hyperliquid demonstrated that an app-specific L1 running its own HotStuff-lineage consensus can host an order book venue with CEX-like latency and dominant perp-DEX market share — the strongest current evidence for the app-chain thesis. Celestia, EigenDA and Avail compete to be everyone’s data layer (Section 4.4).

How to read a TPS claim. Since this section has quoted many throughput figures, it should close with the measurement discipline for evaluating them — a skill this book considers non-optional. Ask, in order: (1) *What is a “transaction”?* Simple transfers inflate counts ~10–50× over DeFi-mix workloads; several chains’ headline numbers historically included validator vote transactions (Solana’s raw TPS figures famously did, roughly 60–70% of the total) or internal system operations. (2) *Sustained or burst? Lab or mainnet?* Firedancer’s 1M TPS was a controlled-environment demonstration; mainnet reality is three orders lower — both numbers are true, and only one is dispositive. (3) *At what verification cost?* A chain “does” X TPS only relative to a specified verifier: anyone claiming high throughput should state the hardware needed to *check* it, which is the decentralisation price actually paid (Section 1.3). (4) *At what finality?* Included-in-400ms is not final-in-400ms (Section 1.1). (5) *Under contention?* Parallel-execution numbers assume conflict-free workloads; ask for the hot-spot benchmark. A comparison that survives all five questions is rare enough that L2Beat, gas-per-second dashboards, and client-team benchmark suites — not marketing pages — are the professional’s sources of record.

The honest meta-picture: *no single architecture has won*. Modular (Ethereum-style) stacks lead in value secured and verification-cost minimisation; integrated (Solana-style) chains

lead in raw UX and composability; app-chains lead in specialised performance. The interesting convergence is that all three now lean on the same two primitives — validity proofs and DAS — exactly as Section 1.2 predicted.

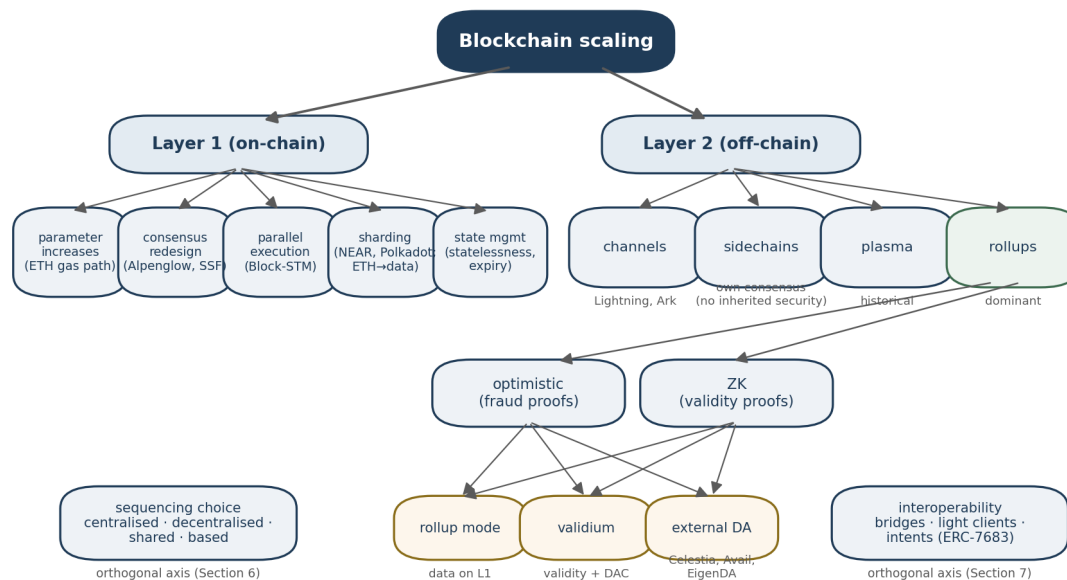


Figure 8.1 — Full-ecosystem taxonomy tree of scaling approaches.

9. Security: attacks and defences in scaled systems

Scaling multiplied the attack surface. Where a monolithic L1 exposes essentially its consensus and its VM, a scaled ecosystem exposes bridges, sequencers, provers, DA committees, upgrade keys, governance, and solvers. This section is organised as case studies with extracted lessons, because in security the particulars are the curriculum. The empirical headline: from 2021 through mid-2026, well over \$3 billion was stolen from cross-chain infrastructure alone, with April 2026 the single worst month in DeFi history (>\$630M across ~30 incidents, including the \$292M Kelp DAO LayerZero-bridge drain and the \$285M Drift social-engineering compromise) — and essentially none of it broke the cryptography. Attackers go around proofs, through keys, committees, and code.

9.1 Bridge exploits: the billion-dollar pattern

Ronin (March 2022, ~\$624M). The Axie Infinity sidechain bridge used a 5-of-9 validator multisig. Lazarus Group (North Korea) spear-phished an engineer with a fake job offer (a trojaned PDF), compromised four Sky Mavis validators, and found a fifth signature in a still-authorized third-party signer from an earlier load-spike arrangement. Withdrawals were forged; the theft went *unnoticed for six days* until a user's failed withdrawal. Lessons: a bridge is exactly as strong as its signer operations; revoke stale authorisations; monitor outflows in real time.

Wormhole (February 2022, ~\$326M). The Solana-side verification contract used a deprecated signature-check instruction that failed to verify the *authenticity of the sysvar account* passed in; the attacker supplied a forged account, “verified” a guardian signature set that never signed, and minted 120k wETH unbacked. Lesson: proof-verification code is consensus-critical code; a single unchecked input equals the entire bridge TVL. (Jump Trading recapitalised the hole — “too big to fail” arriving in DeFi.)

Nomad (August 2022, ~\$190M). A routine upgrade initialised the accepted-root to zero, which made *every* message with a default/empty proof “proven.” Uniquely, the exploit was *permissionless*: after the first attacker, hundreds of copy-paste opportunists replayed the transaction with their own addresses — the first crowd-sourced bridge drain. Lessons: upgrade/initialisation paths are attack surface; property-based tests on “can an unproven message verify?” would have caught it.

Poly Network (August 2021, ~\$611M, returned). Cross-chain manager contract allowed crafted messages to call privileged functions and rotate the keeper keys — arbitrary-call injection through the messaging layer. Lesson: never let the message channel address privileged surface; allowlist targets.

Harmony Horizon (June 2022, ~\$100M): 2-of-5 multisig, keys phished. **BNB Bridge (October 2022, ~\$570M at issue):** forged IAVL Merkle proof accepted by a precompile — again, verification-code soundness. **Orbit, Multichain, Heco (2023–2024):** key compromise and insider control. **2025–2026:** the pattern continues at lower average severity but higher frequency (~\$1.19B in bridge/cross-chain losses in 2024 alone; the 2026 wave described above), with social engineering of privileged operators now the leading initial vector, and North Korean groups responsible for a plurality of value.

The taxonomy that emerges — worth memorising: (1) **verification bugs** (Wormhole, BNB, Nomad): the destination chain accepts an invalid proof; (2) **key/committee compromise** (Ronin, Harmony, Multichain): the trust set is captured, often via phishing; (3) **privileged-surface injection** (Poly): the messaging layer reaches admin functions; (4) **economic/governance capture**: acquire the tokens or seats that control the bridge — demonstrated in pure form by Beanstalk (April 2022, ~\$182M), where an attacker used a flash loan to acquire majority governance power *within a single transaction* and vote through a proposal draining the protocol; any bridge or L2 whose privileged parameters are token-governed without timelocks inherits the same exposure. Defences map one-to-one: minimise and formally verify the verification path (or replace committees with light clients/ZK proofs); harden and monitor signer operations (HSMs, geographic and organisational independence, real-time egress anomaly detection, rate limits and caps per window); isolate messaging from administration; and time-delay + veto every privileged action. Rate limits deserve emphasis as the humble control with the best track record: a bridge that can lose at most \$X per hour turns a catastrophe into an incident.

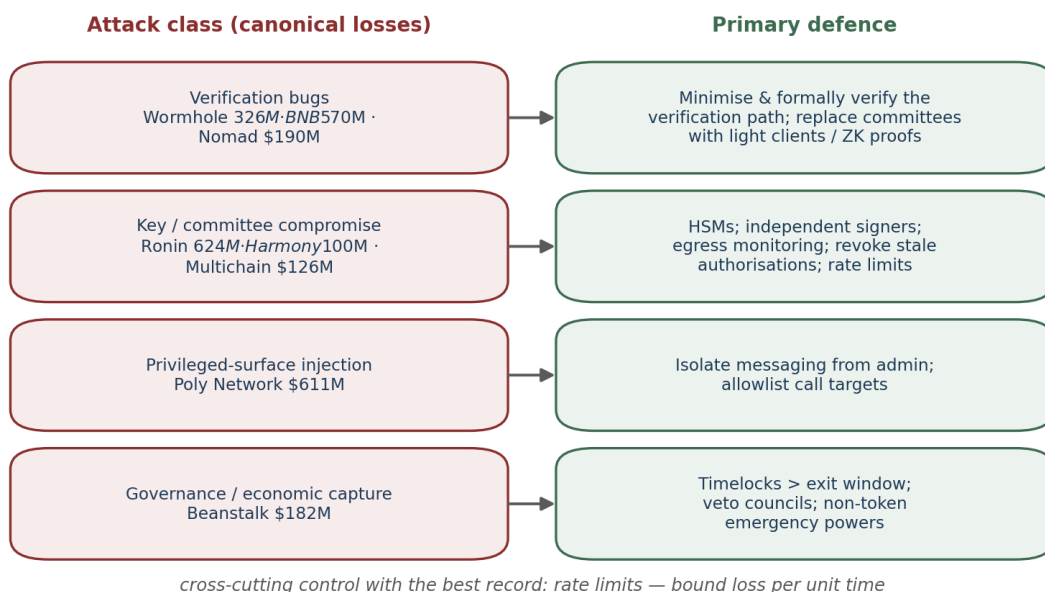


Figure 9.1 — Bridge attack taxonomy mapped to defences.

9.2 Sequencer and liveness failures

Documented earlier (Section 6): Arbitrum 2023/2024, Base 2025 and 2026, plus Solana’s historical full-chain outages (17+ hours, February 2024, its last to date) and Optimism’s early single-sequencer halts. The consistent empirical record — *no user funds lost directly* — validates the forced-inclusion design; the consistent collateral damage — stalled liquidations, frozen market-making, oracle staleness — motivates Section 11’s analysis. Defences: redundant/failover sequencer clusters (imperfectly provisioned failover was itself the cause of Base’s 2025 outage — test your failover), decentralised or based sequencing, short forced-inclusion delays, and application-level circuit breakers that treat “chain dark” as a first-class state.

9.3 Proof-system failures

The soberest category, because it attacks the machinery this entire chapter celebrates.

- **Optimistic side.** In 2022 a whitehat (Jay Freeman) disclosed a critical Optimism bug: a mis-ported self-destruct semantics allowed effectively *infinite ETH minting* on L2 — live for months, unexploited, \$2M bounty. Fraud-proof systems themselves have had soundness gaps disclosed (including in early Cannon iterations and third-party audits of BoLD-era code), all caught pre-exploitation.
- **ZK side.** Multiple production zkEVMs have disclosed *soundness bugs* — circuits that would accept an invalid witness — found by internal teams, auditors, or competitions rather than attackers (examples: publicly documented issues in early Scroll, zkSync-era circuit components, and several zkVM compiler bugs surfaced by the 2025–2026 zkVM security competitions and formal-verification pushes). A soundness bug in a ZK rollup is strictly worse than a fraud-window failure: there is no seven-day pause in which honest parties can react — a forged proof finalises immediately.

- **Verifier-contract risk.** The on-chain verifier is ~500 lines of pairing arithmetic on which billions rest; a mis-encoded verification key or missing input constraint (a class of bug found repeatedly by audits across the industry) voids everything above it.

Defences, in increasing order of ambition: audits and proof-system competitions; *training wheels* (security councils able to pause/override within a bug-response window — the reason Stage 1 exists); **multi-provers** (require 2-of-3 agreement among independently implemented provers — e.g., a SNARK prover, a STARK prover, and a TEE attestation — so a single implementation bug cannot finalise false state; deployed today most fully in the multi-prover configurations of Taiko and planned as *mandatory* in Ethereum’s L1-zkEVM regime); and formal verification of circuits and verifiers (an academic-industrial field growing fast, with the EF’s zkEVM formal-verification project among the flagships).

9.4 Upgrade keys and governance

The quietest systemic risk: most L2 value sits behind *upgradeable* contracts. An attacker (or coerced insider, or captured DAO vote) holding upgrade authority can replace the bridge with a drain — no cryptography violated. Historical near-misses and small realisations abound (Multichain’s opaque key custody ending in a \$126M loss and the arrest of its founder is the cleanest cautionary tale). Defences are the Stage framework’s substance: independent security councils with published membership and quorum rules; timelocks longer than the exit window (Stage 2’s defining requirement) so users can always leave under old rules; on-chain-enforced scope limits on emergency powers; and, at the horizon, native rollups (EIP-9698) that eliminate the largest reason upgrade keys exist at all — tracking EVM changes — by inheriting execution semantics from L1 itself.

9.5 DA-layer and sampling attacks

Data-withholding by a validium DAC (freeze users, extort, or cover fraud); eclipse attacks on DAS samplers (Section 4.3); “lazy committee” risks in restaking-based DA (operators attest without storing — countered by proof-of-custody constructions); and blob-fee griefing (spam the blob market to price rollups out during a critical window — mitigated by BPO-scheduled oversupply). None has produced a headline loss to date; all are actively studied, and the field’s working assumption is that DA attacks will follow value into external-DA systems exactly as bridge attacks followed value into bridges.

9.6 Operator discretion: the Linea precedent, escape hatches, and freezes

A category between attack and defence deserves its own heading: *what operators do with their power during someone else’s attack*. In June 2024, after the Velocore DEX (deployed on Linea) was exploited for ~\$2.6M, the Linea team **paused its sequencer and censored the attacker’s addresses** to stop the stolen funds bridging out. Funds were protected; the episode nonetheless became a landmark, because it demonstrated publicly that a Stage-0 rollup’s “decentralisation roadmap” coexists with an operator who can, today, stop the chain and pick which transactions exist. The team’s own post-incident statement acknowledged the tension and recommitted to decentralising precisely this power. Every architect should be able to argue both sides: censorship-as-defence saved user funds; censorship-as-capability is what Sections 3.6 and 9.4 exist to constrain.

The mirror-image mechanism is the **escape hatch**: a protocol-guaranteed exit that works *against* an uncooperative or dead operator. StarkEx systems pioneered the strongest form — if the operator ignores a forced-withdrawal request for a fixed period, the system enters a *frozen* state in which normal operation halts and users withdraw directly with Merkle proofs of their balances; dYdX v3 users held exactly this guarantee for years, and the mechanism’s existence is why validium users are “frozen, not robbed” under data withholding (Section 4.4) — provided, and this is the examinable caveat, the *data needed for the Merkle proof* is actually obtainable, which is what the DAC attests. Forced-inclusion queues (Section 6) are the rollup-mode analogue. The general design principle: **every promise about user sovereignty must be backed by a code path that works when every operator is adversarial**; anything else is a service-level agreement, not a security property.

Two more entries complete the operator-risk file. Polygon zkEVM’s March 2024 sequencer malfunction halted the chain for ~10 hours (state desynchronisation after an L1 reorg) — routine operationally, but a reminder that ZK rollups’ validity proofs say nothing about liveness. And Blast’s 2023–2024 launch — billions of dollars deposited into what was, for months, a 3-of-5 multisig with no bridge, no rollup, and no exit — demonstrated that the *market* does not price operator risk without institutions like L2Beat forcing disclosure; it reached Stage-classified rollup status only later.

9.7 MEV-adjacent and economic attacks

Sandwiching and liquidation-race dynamics on L2s with public mempools; *time-bandit*-style reorg incentives on fast chains; cross-domain MEV concentrating in solver/sequencer oligopolies; and oracle-latency arbitrage against slow chains (exploited structurally during every major congestion event since 2020 — the March 2020 “Black Thursday” \$0-bid MakerDAO liquidations remain the canonical case: network congestion, not contract error, caused ~\$8M of collateral to clear at zero). Defences: private order flow, batch auctions (CoW-style uniform clearing prices remove intra-batch ordering value), oracle designs with staleness circuit-breakers, and — the deepest fix — enough throughput headroom that congestion-driven failure modes stop being reachable.

10. A programmatic demonstration

Concepts are cheap; code is honest. The following self-contained Python program (~no external dependencies beyond the standard library) demonstrates the chapter’s four load-bearing quantitative ideas: (1) the baseline cost of full replication; (2) rollup batch compression and its fee arithmetic; (3) a fraud-proof bisection game locating a single disputed step in logarithmic rounds; and (4) data availability sampling’s exponential detection guarantee. Students should run it, then modify the parameters (batch sizes, sampling counts, withholding fractions) and confirm the theory’s predictions numerically.

```
#!/usr/bin/env python3
"""
```

```
scaling_demo.py -- quantitative demonstrations of blockchain scaling ideas.
```

1. Full-replication baseline: every node re-executes everything.
 2. Rollup compression: many L2 txs amortised into one L1 commitment.
 3. Fraud proof: interactive bisection finds one bad step in $O(\log n)$.
 4. Data availability sampling: k samples $\rightarrow 2^k$ withholding detection.
 Standard library only. Educational model, not production code.

```

import hashlib
import random
import time
from dataclasses import dataclass, field

H = lambda *parts: hashlib.sha256(b"|".join(
    p if isinstance(p, bytes) else str(p).encode() for p in
    parts)).hexdigest()

# -----
# 1. Baseline: a minimal replicated state machine (account model)
# -----

@dataclass
class Tx:
    sender: str
    recipient: str
    amount: int
    nonce: int

class Chain:
    """Every node runs this same logic on every tx: total replication."""
    def __init__(self, balances):
        self.state = dict(balances)          # account -> balance
        self.nonces = {a: 0 for a in balances}
        self.blocks = []

    def state_root(self):
        # Commitment to state: hash of sorted (account, balance, nonce).
        leaves = sorted((a, self.state[a], self.nonces[a]) for a in
self.state)
        return merkle_root([H(*leaf) for leaf in leaves])

    def apply(self, tx: Tx) -> bool:
        if self.nonces.get(tx.sender, 0) != tx.nonce:          return False
        if self.state.get(tx.sender, 0) < tx.amount or tx.amount <= 0: return
False
        self.state[tx.sender] -= tx.amount
        self.state[tx.recipient] = self.state.get(tx.recipient, 0) +
tx.amount
        self.nonces[tx.sender] += 1
        self.nonces.setdefault(tx.recipient, self.nonces.get(tx.recipient,

```

```

0))

    return True

def produce_block(self, txs):
    applied = [tx for tx in txs if self.apply(tx)]
    root = self.state_root()
    self.blocks.append({"n": len(applied), "state_root": root})
    return root

def merkle_root(leaves):
    if not leaves: return H("empty")
    layer = leaves[:]
    while len(layer) > 1:
        if len(layer) % 2: layer.append(layer[-1])
        layer = [H(layer[i], layer[i+1]) for i in range(0, len(layer), 2)]
    return layer[0]

def demo_replication(n_nodes=1000, n_txs=5000):
    accounts = {f"acct{i}": 1_000_000 for i in range(100)}
    txs = [Tx(f"acct{i%100}", f"acct{(i*7)%100}", 10, i//100) for i in range(n_txs)]
    t0 = time.perf_counter()
    node = Chain(accounts); node.produce_block(txs)
    per_node = time.perf_counter() - t0
    print("== 1. Full replication baseline ==")
    print(f" one node executed {n_txs} txs in {per_node*1e3:.1f} ms")
    print(f" network of {n_nodes} nodes performs {n_nodes} x that work")
    print(f" aggregate work: {n_nodes*per_node:.1f} node-seconds for "
          f"{n_txs} txs -> throughput == one node's throughput.\n")

# -----
# 2. Rollup: execute off-chain, commit data + state root on-chain
# -----

TX_BYTES_L1    = 112    # approx signed transfer size posted individually
TX_BYTES_L2    = 12     # compressed in-batch encoding (to/amount/nonce deltas)
BATCH_OVERHEAD = 300    # state root + proof/commitment overhead per batch
BLOB_BYTE_COST = 0.000002 # illustrative $ per byte of blob data

def demo_rollup(n_txs=5000, batch=1000):
    accounts = {f"acct{i}": 1_000_000 for i in range(100)}
    l2 = Chain(accounts) # sequencer executes off-chain
    l1_posted = [] # what actually hits L1
    txs = [Tx(f"acct{i%100}", f"acct{(i*7)%100}", 10, i//100) for i in range(n_txs)]
    for i in range(0, n_txs, batch):
        chunk = txs[i:i+batch]
        root = l2.produce_block(chunk)
        data = len(chunk)*TX_BYTES_L2 + BATCH_OVERHEAD

```



```

        l1_posted.append({"bytes": data, "state_root": root,
                          "data_hash":
H(*[H(t.sender,t.recipient,t.amount,t.nonce)
                                for t in chunk]))}

naive = n_txs * TX_BYTES_L1
rolled = sum(b["bytes"] for b in l1_posted)
print("== 2. Rollup compression ==")
print(f" {n_txs} txs individually on L1: {naive:,} bytes")
print(f" as {len(l1_posted)} rollup batches: {rolled:,} bytes "
      f"({naive/rolled:.1f}x compression)")
print(f" illustrative DA cost/tx: ${rolled*BLOB_BYTE_COST/n_txs:.6f}")
print(f" final L2 state root committed on L1: {l1_posted[-
1]['state_root'][:16]}...\n")

# -----
# 3. Optimistic fraud proof: interactive bisection
# -----

def execution_trace(chain: Chain, txs):
    """List of state roots after each tx: the 'trace' both parties argue
    over."""
    roots = [chain.state_root()]
    for tx in txs:
        chain.apply(tx); roots.append(chain.state_root())
    return roots

def demo_fraud_proof(n_txs=1024, bad_index=613):
    accounts = {f"acct{i}": 1_000_000 for i in range(100)}
    txs = [Tx(f"acct{i%100}", f"acct{(i*3)%100}", 5, i//100) for i in
range(n_txs)]
    honest = execution_trace(Chain(accounts), txs)
    fraud = honest[:] # cheating sequencer copies...
    for i in range(bad_index+1, len(fraud)): # ...then corrupts from
bad_index on
        fraud[i] = H("stolen-funds", i)
    lo, hi, rounds = 0, n_txs, 0 # L1 contract's view: dispute
[lo,hi)
    while hi - lo > 1:
        mid = (lo + hi)//2; rounds += 1
        # challenger agrees with claimed root at mid? honest trace is ground
truth
        if fraud[mid] == honest[mid]: lo = mid
        else: hi = mid
    # single disputed step: L1 re-executes ONLY tx[lo]
    check = Chain(accounts)
    for t in txs[:lo]: check.apply(t)
    check.apply(txs[lo])
    verdict = "FRAUD PROVEN" if check.state_root() != fraud[lo+1] else "claim
upheld"
    print("== 3. Fraud proof by bisection ==")

```

```

    print(f"  trace length {n_txs}; corrupted from step {bad_index}")
    print(f"  bisection converged in {rounds} rounds
(log2({n_txs})={n_txs.bit_length()-1})")
    print(f"  L1 re-executed exactly 1 tx of {n_txs}: {verdict}\n")

# -----
# 4. Data availability sampling
# -----

def demo_das(chunks=256, withheld_frac=0.5, max_samples=30, trials=20000):
    """Erasure-coded block of 2n chunks; adversary withholds a fraction.
    Sampler asks for k random chunks; detection = any sample missing."""
    print("== 4. Data availability sampling ==")
    n_ext = 2*chunks
    hidden = set(random.sample(range(n_ext), int(n_ext*withheld_frac)))
    print(f"  extended chunks: {n_ext}; withheld: {len(hidden)} "
          f"({withheld_frac:.0%}, the minimum that can hide any data)")
    print(f"  {'k':>3} {'P(miss) theory':>15} {'P(miss) simulated':>18}")
    for k in (1, 2, 5, 10, 20, max_samples):
        theory = (1 - withheld_frac)**k
        misses = sum(all(random.randrange(n_ext) not in hidden for _ in
range(k))
                     for _ in range(trials))
        print(f"  {k:>3} {theory:>15.2e} {misses/trials:>18.2e}")
    print(f"  30 samples of ~2 KB each verify a ~{n_ext*2/1024:.0f} MB
extended block "
          f"with ~1e-9 failure probability.\n")

# -----
# 5. Inheriting L1 security: the abandonment test
#   Rollup: batch DATA on L1 -> anyone can rebuild state and exit.
#   Sidechain: only signed roots on L1 -> no data, no proof, no exit.
# -----

def merkle_root_and_proof(leaves, index):
    """Root plus inclusion proof [(sibling, sibling_is_right), ...]."""
    layer, idx, path = leaves[:], index, []
    while len(layer) > 1:
        if len(layer) % 2: layer.append(layer[-1])
        sib = idx + 1 if idx % 2 == 0 else idx - 1
        path.append((layer[sib], sib > idx))
        layer = [H(layer[i], layer[i+1]) for i in range(0, len(layer), 2)]
        idx //= 2
    return layer[0], path

def merkle_verify(root, leaf_hash, path):
    h = leaf_hash
    for sib, sib_right in path:
        h = H(h, sib) if sib_right else H(sib, h)

```

```

    return h == root

def state_leaves(chain):
    return sorted((a, chain.state[a], chain.nonces[a]) for a in chain.state)

def demo_l1_inheritance(n_txs=2000, batch=500, alice="acct7"):
    print("== 5. The abandonment test: what 'inheriting L1 security' means ==")
    genesis = {f"acct{i}": 1_000_000 for i in range(100)}
    txs = [Tx(f"acct{i%100}", f"acct{(i*11)%100}", (i % 7) + 1, i//100)
            for i in range(n_txs)]

    # --- normal operation: operator runs both systems, posts to L1 -----
    rollup, sidechain = Chain(genesis), Chain(genesis)
    L1_rollup, L1_side = [], [] # what each design writes to L1
    for i in range(0, n_txs, batch):
        chunk = txs[i:i+batch]
        L1_rollup.append({"data": chunk, # <-- the difference:
                          "root": rollup.produce_block(chunk)}) # data ON
L1
        L1_side.append({"root": sidechain.produce_block(chunk)}) # root only
    print(f" operator posted {len(L1_rollup)} batches, then VANISHED.")

    # --- rollup path: anyone rebuilds state from L1 data alone -----
    replay = Chain(genesis) # public code + L1
data
    for b in L1_rollup:
        for t in b["data"]: replay.apply(t)
    ok_root = replay.state_root() == L1_rollup[-1]["root"]
    leaves = state_leaves(replay)
    idx = [l[0] for l in leaves].index(alice)
    bal = leaves[idx][1]
    root, proof = merkle_root_and_proof([H(*l) for l in leaves], idx)
    accepted = (root == L1_rollup[-1]["root"] and
                merkle_verify(L1_rollup[-1]["root"], H(*leaves[idx]), proof))
    print(f" [rollup] state rebuilt from L1 data: matches posted root:
{ok_root}")
    print(f" [rollup] {alice} proves balance {bal:,} with a "
          f"{len(proof)}-hash Merkle proof -> bridge releases funds:
{accepted}")

    # --- sidechain path: no data on L1, nothing to prove -----
    print(f" [sidechain] L1 holds only bare roots "
          f"({L1_side[-1]['root'][:12]}...): state CANNOT be rebuilt;")
    print(f" [sidechain] no inclusion proof can be formed; bridge answers to
a")
    print(f" vanished multisig -> funds unrecoverable.\n")

if __name__ == "__main__":

```

```

random.seed(42)
demo_replication()
demo_rollup()
demo_fraud_proof()
demo_das()
demo_l1_inheritance()

```

Representative output (exact timings vary by machine):

```

== 1. Full replication baseline ==
one node executed 5000 txs in 3.8 ms
network of 1000 nodes performs 1000 x that work
aggregate work: 3.8 node-seconds for 5000 txs -> throughput == one node's.

```

```

== 2. Rollup compression ==
5000 txs individually on L1: 560,000 bytes
as 5 rollup batches: 61,500 bytes (9.1x compression)
illustrative DA cost/tx: $0.000025
final L2 state root committed on L1: 28d09cc720f608ba...

```

```

== 3. Fraud proof by bisection ==
trace length 1024; corrupted from step 613
bisection converged in 10 rounds (log2(1024)=10)
L1 re-executed exactly 1 tx of 1024: FRAUD PROVEN

```

```

== 4. Data availability sampling ==
extended chunks: 512; withheld: 256 (50%)
  k  P(miss) theory  P(miss) simulated
  1      5.00e-01      4.99e-01
  5      3.12e-02      3.20e-02
 10      9.77e-04      9.00e-04
 30      9.31e-10      0.00e+00
30 samples of ~2 KB each verify a ~1 MB extended block with ~1e-9 failure
probability.

```

```

== 5. The abandonment test: what 'inheriting L1 security' means ==
operator posted 4 batches, then VANISHED.
[rollup]   state rebuilt from L1 data: matches posted root: True
[rollup]   acct7 proves balance 1,000,005 with a 7-hash Merkle proof
           -> bridge releases funds: True
[sidechain] L1 holds only bare roots (416c5b91b645...): state CANNOT be
rebuilt;
           no inclusion proof can be formed; bridge answers to a
           vanished multisig -> funds unrecoverable.

```

Five observations tie the code back to the chapter. The replication demo makes the trilemma's origin tangible: adding nodes adds security and *nothing else*. The rollup demo shows that compression ratios (here ~9x; production compression plus blob pricing yields far more) — not faster execution — are where L2 fee reductions come from, and that the state root, twelve bytes of it shown, is the entire on-chain footprint of correctness. The

bisection demo is the Arbitrum insight in twenty lines: the L1 adjudicates a million-step dispute by executing *one* step. The DAS demo is the reason PeerDAS works: the defender's task (hide $\geq 50\%$ of chunks) collides with the attacker's exposure (each sample is a coin flip), and thirty coin flips settle the matter to one-in-a-billion. The abandonment-test demo executes Section 3.4's inheritance argument literally: the *only* difference between its rollup and its sidechain is one field — whether data was written to L1 alongside the root — and that single field is the difference between a seven-hash Merkle proof releasing Alice's funds and an unrecoverable balance; students should delete the "data" key and watch the recovery path fail, then re-derive why every architecture in Table 12.1 sits where it does on the exit column. Suggested exercises: make the fraud-proof defender adversarial (answering bisection queries dishonestly) and show the game still converges; add per-chunk Merkle proofs to the DAS demo; model an eclipse attack by letting the adversary answer *this sampler's* queries selectively.

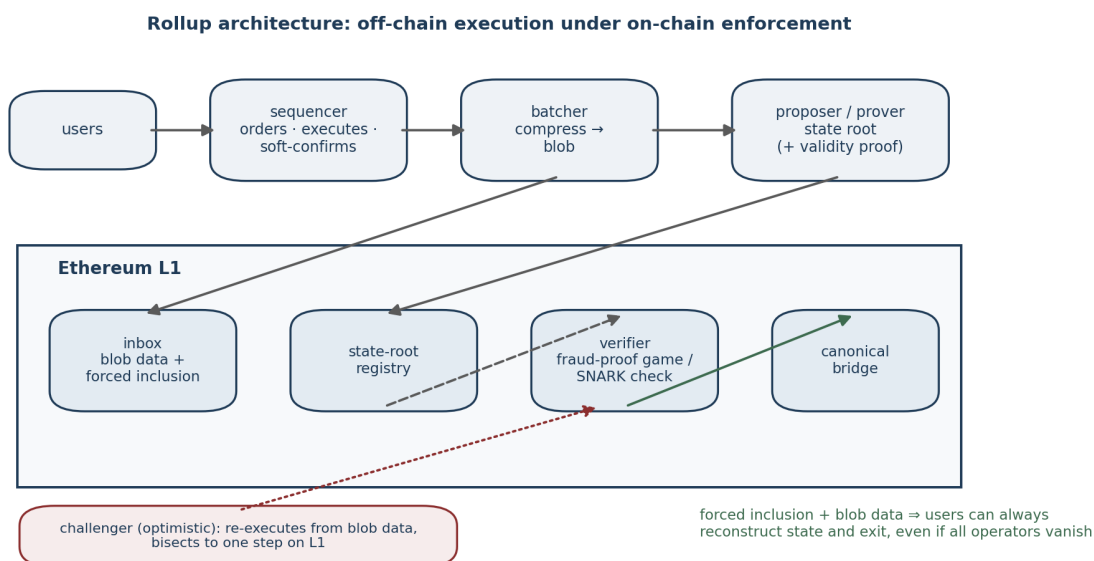


Figure 10.1 — Rollup architecture and fraud-proof/validity-proof flow.

11. Scalability in practice: DeFi and TradFi

11.1 DeFi: what scaling enables and what it breaks

DeFi is the most demanding customer of blockchain throughput, because its core mechanisms are *latency- and congestion-sensitive in ways that turn performance problems into solvency problems*:

- **Liquidations.** Lending protocols (Aave, Compound lineage) stay solvent only if undercollateralised positions are liquidated promptly. Congestion delays liquidations and inflates the gas price of executing them; Black Thursday (March 2020) showed the endgame — liquidation auctions clearing at \$0 bids because competing keepers could not land transactions. Scaled venues shrink this risk (cheap blockspace means

liquidation is always profitable to execute) but add a new one: an L2 *outage* suspends liquidation entirely while collateral prices move on CEXs (Section 9.2). Mature protocols now treat sequencer downtime as a first-class oracle condition — Chainlink’s L2 sequencer-uptime feeds let contracts pause liquidations and new borrowing during an outage and apply grace periods after recovery, a defence invented after, and directly because of, early L2 outages.

- **Oracles.** Cheap blockspace changed oracle architecture: push-based feeds updating on deviation thresholds (viable when updates cost dollars) are giving way to pull-based/low-latency designs (Pyth-style) on scaled chains, narrowing the oracle-latency arbitrage that congestion used to widen.
- **AMMs vs order books.** Order books were priced off Ethereum L1 by gas (Uniswap’s constant-product AMM was, at bottom, a gas-cost-driven design compromise of genius); scaling reversed the trade. dYdX left Ethereum for its own Cosmos app-chain (v4, 2023) for order-book latency; Hyperliquid’s app-L1 order book took dominant perp share by 2025–2026; Uniswap itself launched Unichain (an OP-stack rollup with fast blocks and MEV-aware sequencing) — the flagship application choosing to *become* infrastructure. The lesson runs both directions: scaling enables CEX-like DeFi, and DeFi’s requirements now drive scaling design (fast preconfirmations, private order flow, sub-second finality).
- **Congestion cascades: a worked example.** The failure modes above compound. During the USDC depeg weekend (March 2023), redemption panic drove L1 gas above 300 gwei; stablecoin-pool imbalances triggered liquidation and rebalancing waves; keepers bidding against each other in priority-gas auctions pushed fees higher still, which delayed *other* protocols’ liquidations, widening their bad debt — a positive-feedback loop in which congestion begets urgency begets congestion. Every major congestion event since 2020 (Black Thursday, the 2021 NFT mints, inscription waves on multiple chains in 2023–2024) exhibits the same dynamics. The systemic-risk lesson is that blockspace scarcity acts procyclically — demand for inclusion is highest exactly when the system can least accommodate it — so *headroom is a safety property, not a luxury*, and stress-period capacity, not average capacity, is the number that matters in protocol risk models.
- **Composability under fragmentation.** L1 DeFi’s superpower — atomic composability across every protocol in one transaction (flash loans as the purest expression) — does not survive the move to many L2s. Cross-rollup DeFi is asynchronous: solver-mediated, intent-based, and settlement-risk-bearing (Section 7). Whether atomic cross-rollup composability can be recovered (shared sequencing, based rollups with same-slot inclusion, AggLayer-style unified settlement) is among the field’s most commercially significant open problems.

11.2 TradFi: tokenisation meets throughput

Traditional finance’s engagement with public chains crossed from pilots to production during 2024–2026, and scalability was a precondition:

- **Stablecoins and payments.** Stablecoin supply above \$250B and annual transfer volume in the tens of trillions of dollars made stablecoins the first blockchain product

with unambiguous product-market fit; the US GENIUS Act (2025) and the EU's MiCA gave issuance a regulatory frame. Payment-grade UX (sub-cent fees, sub-second confirmation) exists *only* on scaled infrastructure — L2s, Solana, and dedicated payment chains (including L1s launched by payment incumbents and stablecoin issuers themselves: Stripe's Tempo, Circle's Arc, Tether-aligned Plasma and Stable). Visa-scale peak load (~65k TPS) remains beyond any single production chain's *sustained, verifiable* throughput but is within reach of aggregate L2 capacity on current roadmaps — the first time that sentence has been true.

- **Tokenised funds and RWAs.** BlackRock's BUIDL (tokenised money-market fund, 2024, then multi-chain), Franklin Templeton's BENJI, and tokenised Treasury products collectively crossed the tens of billions; tokenised-RWA totals passed \$25–30B by 2026. These workloads need modest throughput but *strict* finality, compliance hooks, and — increasingly — privacy, which pushes institutions toward validium-style designs and ZK disclosure rather than public rollups: an instructive case of TradFi selecting a *different point* on the DA/trust spectrum of Section 4.4 than DeFi does.
- **Exchange and settlement infrastructure.** DTCC (which settles quadrillions annually) has run tokenised-collateral pilots (Project Ion lineage, AppChain work); SWIFT connected its network to chain transfer trials; the ECB and BIS (Project Agorá) advanced wholesale-CBDC and unified-ledger experiments. The institutionally revealing detail: these systems consistently prioritise deterministic finality, permissioned validation, and recoverability over decentralisation — they are trilemma-corner choices made deliberately, and they interoperate with public chains at asset level (tokenised funds) rather than protocol level.
- **The convergence workload.** The demanding hybrid — 24/7 tokenised equities and funds trading against stablecoins on public rollups with institutional compliance — is the load test the 2027-era stack (real-time proving, 48-blob DA, intent-based interop, privacy-preserving disclosure) is being built to pass.

What TradFi actually requires of a scaling stack. Synthesising the institutional pilots, the requirements list differs instructively from DeFi's: *deterministic finality* with a legally citable timestamp (probabilistic and challenge-window finality complicate settlement law — one reason ZK rollups and BFT chains recur in institutional selections); *operational recoverability* (a mis-sent securities transfer must be reversible by legal process, pulling institutions toward permissioned validation or administrative controls that public-chain culture resists); *identity and disclosure hooks* (transfer-agent and AML duties met increasingly by ZK credentials rather than public transparency); *throughput ceilings that are boring* (a settlement system must be sized for triple its worst historical day, per the headroom lesson above); and *decade-scale continuity* (a fund share is a 30-year instrument; Section 13's post-quantum and state-growth problems are, for institutions, present-tense procurement questions). Scalability, in the TradFi frame, is one line item in an operational-resilience checklist — a useful corrective to the TPS-first framing that dominates the crypto-native discourse.

11.3 Institutions and Layer 2: use, build, or bypass?

The previous subsections described what institutions require of scaling infrastructure; this one examines the strategic question they actually face — *which* infrastructure — because between 2024 and 2026 that question moved from innovation-lab speculation to board-level procurement, and the answers given by named institutions now form a substantial empirical record. The decision space has three branches: use an existing public L2, build (or commission) your own, or bypass the Ethereum stack entirely for purpose-built institutional rails, of which the Canton Network is the most consequential example. Each branch has been taken by at least one major bank, and the pattern of who chose what is itself instructive.

Why an institution needs an L2 at all. Start from the requirements of Section 11.2 and ask what happens when a regulated institution contemplates transacting on public Ethereum L1. Four problems appear immediately. *Privacy*: L1 transparency is a non-starter for client flows — a bank cannot publish its clients' positions, and pre-trade transparency of pending transactions invites front-running of institutional-size orders (the MEV problem of Section 6, which for a bank is not an inefficiency but a fiduciary breach). *Compliance control*: an institution must be able to enforce KYC/AML at the point of transfer, screen counterparties against sanctions lists, and demonstrate to supervisors that it can — none of which base-layer Ethereum provides. *Cost and throughput determinism*: L1 fee volatility is incompatible with product pricing. *Administrative capability*: as Section 11.2 noted, securities law effectively requires the power to reverse, freeze, or reissue under legal process. An L2 — where the institution or its vendor controls the execution environment, the participant set, and the privacy envelope, while the base layer supplies settlement assurance, timestamping, and interoperability — is the natural place to reconcile these requirements with the benefits that drew the institution to public-chain technology in the first place: shared infrastructure, 24/7 operation, atomic delivery-versus-payment, and access to a large liquidity and developer ecosystem. In terms of this chapter's framework: institutions want the *stratification* of Section 1.3, but with the additional twist that they often want to *hold* some of the operator powers that crypto-native designs work to eliminate. The abandonment test of Section 3.4 is deliberately, and honestly, *not* the institutional design goal — recoverability under legal process is valued above unconditional user sovereignty — and a well-run institutional programme states this trade openly rather than obscuring it.

Branch 1: use an existing public L2. The landmark here is J.P. Morgan's Kinexys division, which piloted JPMD — a USD *deposit token*, i.e., tokenised commercial bank money, legally distinct from a stablecoin — on Coinbase's Base in June 2025 and moved it into production under the JPM Coin brand in November 2025, making J.P. Morgan the first major bank to issue a dollar claim on a public blockchain, with B2C2, Mastercard and Coinbase among early transactors. The logic of choosing a public L2 is *distribution*: Base's users, liquidity, and integrations are already there, the chain's operations are someone else's problem, and the bank's compliance perimeter travels with the token itself (JPMD transfers are restricted to allowlisted institutional clients at the contract level — compliance enforced in the asset, not the chain). UBS ran the same logic in proof-of-concept form, testing its Key4 Gold retail gold product on a ZKsync validium — the off-chain-data mode of Section 4.4 chosen

explicitly for privacy and cost. The cautions mirror Section 9: the bank inherits the L2 operator's sequencer liveness (a Base outage is now a J.P. Morgan service outage), its upgrade governance, and its Stage classification, and public-chain MEV and transparency still constrain which workloads fit. It is telling that deposit-token *transfers* went to a public L2 while the banks' *interbank settlement* stayed elsewhere.

Branch 2: build your own. Deutsche Bank's Project Dama 2 — developed with Memento Blockchain and Interop Labs on ZKsync's ZK Stack, announced December 2024 with an institutional "litepaper" published in June 2025 — is the clearest articulated case of a bank *building* an Ethereum L2: a "public-permissioned" validium-style chain that settles to Ethereum, but on which the bank selects validators, controls participation, and — the detail that attracted the most commentary — can provision regulators with "super admin" visibility and intervention rights. The stated use cases are tokenised fund issuance and servicing, digital-asset/digital-cash interoperability, and near-real-time settlement. The adjacent corporate precedents show the same pattern outside banking: Sony's Soneium (OP Stack, mainnet January 2025), Kraken's Ink (OP Stack), and Robinhood Chain (Arbitrum Orbit; mainnet mid-2026, purpose-built for 24/7 tokenised stocks and described by the firm as its largest product initiative) all demonstrate that launching a branded L2 on a commodity rollup stack is now an ordinary corporate infrastructure decision — Section 13's practitioner checklist item "choose boring where possible" made real. The *pros* of building: full control of sequencing (no public mempool, hence no third-party MEV against client flow — the sequencer-as-compliance-perimeter), fee and admission policy, privacy architecture (validium mode with the institution as its own DA committee), upgrade cadence aligned to regulatory approval, and capture of the sequencer economics that Branch 1 cedes (Section 3.5's revenue discussion — material at Base-like scale). The *cons* are the entire operational catalogue of Section 9 transferred onto the institution's own balance sheet and audit scope: sequencer liveness, prover soundness, key ceremony and upgrade governance, plus two strategic costs — *liquidity fragmentation* (a private chain has no organic counterparties; every asset must be bridged or issued twice) and *credible-neutrality loss* (counterparties must trust the operating bank's chain governance, which is precisely the objection interbank infrastructure exists to avoid — one reason single-bank L2s skew toward *servicing* workloads rather than interbank markets).

Branch 3: bypass — Canton and the consortium rails. The question "does Canton help?" deserves a precise answer, because Canton is the strongest institutional counter-thesis to the Ethereum L2 stack, and it is frequently mis-described. The Canton Network (Digital Asset, launched 2023–2024, built on the Daml smart-contract language) is *not* an Ethereum L2 and inherits nothing from Ethereum; it is a "network of networks" of permissioned application chains that can synchronise atomic transactions across one another through a shared coordination service (the Global Synchronizer, operated by a federation of "super validator" institutions under the Global Synchronizer Foundation, with Canton Coin as its fee/incentive asset). Its defining property is *sub-transaction privacy*: participants see only the parts of a transaction they are party to — contrast this with a public rollup, where privacy must be bolted on. On the workloads institutions care most about, its production record is currently ahead of any public-chain venue: Broadridge's Distributed Ledger Repo, running on Canton rails, processes on the order of \$4 trillion in *monthly* intraday/overnight

Treasury repo; DTCC has partnered to make DTC- and Fed-eligible collateral usable on the network; Goldman Sachs's GS DAP tokenisation platform is built on the same technology; Société Générale-FORGE has brought its EURCV/USDCV stablecoins to it; and in January 2026 Kinexys announced intent to issue JPM Coin natively on Canton — notable because it means the same bank money now runs on Branch 1 (Base) and Branch 3 simultaneously, one instance per workload. Digital Asset's 2025 raise (led by DRW with Goldman, Citadel Securities and others) and reported network totals in the trillions of dollars of represented assets should be read with the usual scepticism about self-reported figures, but the direction is unambiguous. *How* Canton helps: it delivers, natively, exactly the four properties Branch 1 and 2 struggle to retrofit — privacy, permissioning, atomic cross-application settlement without global transparency, and legally conventional operator accountability. *What it does not deliver* is equally clear from this chapter's framework: there is no trust-minimised inheritance in the Section 3.4 sense (validation is by the parties and named operators; the abandonment test is not the design goal and would fail — deliberately); there is no access to public-chain liquidity, DeFi composability, or the credible neutrality of a validator set no consortium member controls; and the network effect is gated to institutions that adopt the Daml stack. The same trade, in narrower vertical slices, defines the rest of the consortium field: Finality (owned by a bank consortium; its Sterling Finality Payment System became the first regulated DLT wholesale payment system settling in central-bank-backed digital cash, live December 2023, UK settlement-finality designation December 2024, \$136M Series C in 2025, and 2026 delivery-versus-payment settlements of digital gilts with Lloyds); Partior (JPMorgan/DBS/Temasek/Standard Chartered-founded multicurrency interbank clearing, expanding through 2025–2026); and SWIFT's shared ledger (announced with a 17-bank pilot cohort including Citi, HSBC and UBS in July 2026 — an *orchestration* layer recording payment commitments between tokenised-deposit systems that remain on the banks' own ledgers). None of these is an L2; all of them compete with L2s for the same institutional workloads, and the honest comparison is trust-model versus network-access, not throughput.

The synthesis, and the pattern in the record. Laid against Section 3.6's three questions, the branches order cleanly: a public L2 maximises external network access while importing another operator's governance; an own-build maximises control while importing all operational risk and forfeiting neutrality; Canton and the consortium rails maximise privacy and legal conventionality while forfeiting public-chain access entirely. The empirical pattern of 2024–2026 is that major institutions are refusing to choose *once*: the same institution runs distribution-facing products on public L2s (JPMD on Base), servicing infrastructure on controlled chains (Dama 2), and interbank settlement on consortium rails (Canton, Finality, Partior, SWIFT) — a portfolio, allocated per workload by exactly the trust/throughput/privacy calculus this chapter has developed, with EY's open-sourced Nightfall_4 (an enterprise ZK rollup for private transfers on public Ethereum, its optimistic design replaced by validity proofs in 2025) marking the technical frontier where the branches may eventually reconverge: public-chain settlement with institution-grade privacy. For the architect advising an institution, the disciplined questions are Section 4.5's, plus three institutional additions: which regulator must be satisfied, and what visibility does it require; which counterparties must trust the chain, and will they accept your

control of it; and which workloads genuinely need public liquidity — because every workload that does not will, on current evidence, end up somewhere permissioned.

Table 11.1 — Institutional deployments on record (2023–2026)

Institution / initiative	Approach	Stack / venue	Status (mid-2026)
J.P. Morgan (Kinexys) — JPM Coin/JPMD	deposit token on public L2	Base (OP Stack)	production, Nov 2025; Canton issuance announced Jan 2026
Deutsche Bank — Project Dama 2	own public-permissioned L2	ZKsync ZK Stack (validium-style)	litepaper Jun 2025; MVP subject to regulatory approval
UBS — Key4 Gold	pilot on public-stack validium	ZKsync Validium	proof-of-concept
Robinhood — Robinhood Chain	own L2 for tokenised stocks	Arbitrum Orbit	mainnet 2026
Sony — Soneium; Kraken — Ink	corporate L2s	OP Stack	mainnets 2025
Broadridge DLR; DTCC; Goldman GS DAP; SG-FORGE	consortium rail (not an L2)	Canton Network (Daml)	production; ~\$4T/month repo via DLR
Finality (£FnPS)	bank-owned wholesale payment system	proprietary DLT, central-bank-backed cash	live Dec 2023; UK settlement-finality designation 2024
Partior	interbank multicurrency clearing	proprietary DLT	live, expanding 2025–2026
SWIFT shared ledger	orchestration of tokenised deposits	proprietary ledger	17-bank pilot cohort, Jul 2026
EY — Nightfall_4	open-source privacy ZK rollup on Ethereum	public domain	released 2025

12. Comparative evaluation

The following tables consolidate the chapter. (Ratings are qualitative; numbers approximate, mid-2026.)

Table 12.1 — Scaling families compared

Approach	Security inherited	Throughput gain	Latency/finality	Data on L1?	Main trust	Canonical examples
----------	--------------------	-----------------	------------------	-------------	------------	--------------------

	from base?				assumpti on	
L1 parame ter increas e	n/a (is base)	~2–10×	unchanged	yes	node- resource creep	ETH gas-limit path, BCH
Consen sus redesig n	n/a	~1–5× (latency >>)	ms–s finality	yes	validator set	Alpenglow, HotStuff-family
Parallel executi on	n/a	~5–50× (workloa d-dep.)	unchanged	yes	hot-spot contentio n	Solana, Monad, Aptos/Sui
Executi on shardin g	partial– full	~linear in shards	cross-shard async	per-shard	committe e sampling	NEAR, Polkadot
State channel s	full (for participa nts)	~unboun ded off- chain	instant	no (commitm ents only)	liveness + watchto wers	Lightning
Sidecha in	no	high	fast	no	its validator s + bridge	Polygon PoS (hist.), Ronin
Plasma	partial	high	slow exits	commitme nts only	operator DA	(historical)
Optimis tic rollup	yes (state validity)	~10– 100×	fast soft; 7d exits	yes	1 honest challenge r + DA	Arbitrum, OP, Base
ZK rollup	yes (state validity)	~10– 100×	fast; h-scale exits	yes	proof soundnes s + DA	Starknet, ZKsync, Scroll, Linea
Validiu m	validity only	~100– 1000×	fast	no	DAC/DA layer honesty	StarkEx-family, many perp DEXs
Externa l DA + rollup	partial	~100×+	fast	no (on DA layer)	DA layer consensu s	Celestia/EigenDA /Avail rollups

Table 12.2 — Optimistic vs ZK rollups

Dimension	Optimistic	ZK
-----------	------------	----

Validity mechanism	fraud proofs, 1-of-N honest + censorship-resistant L1	validity proofs, cryptographic soundness
Withdrawal latency	~7 days (or trusted fast-exit LPs)	minutes–hours (proof cadence)
EVM fidelity	trivially high (re-execution)	solved via zkVMs; Type 1 now practical
Operating cost	low fixed; high only under dispute	proving cost per batch (falling fast; GPU fleets)
Failure mode	missed challenge window; verifier apathy	soundness bug finalises instantly
2026 trajectory	adding ZK validity as fast path (hybrid)	adding fraud-proof/multi-prover fallbacks (hybrid)

Table 12.3 — DA options (security ↓, throughput/cost-efficiency ↑ as you read downward)

Option	Guarantee	Approx. capacity (2026)	Trust
Ethereum blobs + PeerDAS	consensus-sampled public DA	stepping toward 48 blobs/block (~0.75 MB/s and rising)	Ethereum validators
Avail	DAS + KZG validity-proved encoding	4 MB blocks → multi-GB roadmap	Avail validators
Celestia	DAS + fraud-proved encoding	8→128 MB blocks (Matcha); GB-scale roadmap	Celestia validators
EigenDA	attested custody (restaked)	~100 MB/s (v2)	operator quorum + slashing
DAC	committee signatures	arbitrary	n-of-m named entities

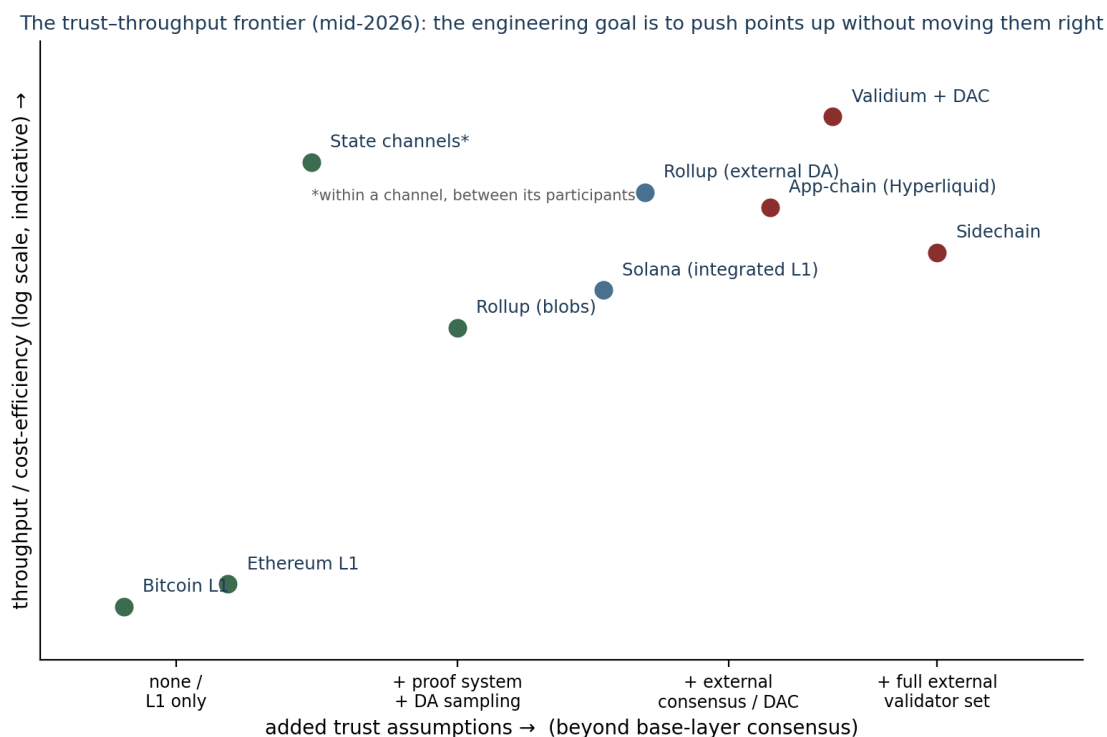


Figure 12.1 — Trust-vs-throughput frontier of DA and execution choices.

A practitioner’s checklist. The tables compress into a decision procedure that this author has found robust in architecture reviews. First, *fix the security floor*: what is the maximum value the system will secure, and is there any acceptable scenario in which a named committee can freeze or take it? If not, the DA and proof choices are already made (L1 DA, validity or mature fraud proofs, Stage-1-or-better governance), and the remaining freedom is sequencing and stack choice. Second, *size the workload honestly*: transactions per second at the 99th-percentile stress day, bytes per transaction after compression, latency the product genuinely requires (sub-second UX needs preconfirmations or an integrated chain; nothing in lending needs it). Third, *price the exits*: compute, for a representative user, the cost and time to leave the system with every operator adversarial — if the answer is “cannot,” the design is a custodial service with extra steps, whatever its architecture diagram says. Fourth, *choose boring where possible*: shared stacks (OP Stack, Arbitrum Orbit, ZK stacks with shared provers) inherit audited code, fault-proof machinery, and an upgrade community; bespoke infrastructure is justified only by requirements the shared stacks demonstrably cannot meet (Hyperliquid’s order book being the honest example). Fifth, *write the risk disclosure first*: if the one-paragraph statement of who can do what to users is embarrassing to publish, the architecture is wrong.

13. Open research problems

An honest map of what remains unsolved, phrased as researchable questions:

1. **Decentralised sequencing with fair ordering.** Can a sequencer set provide sub-second preconfirmations, MEV resistance (or fair MEV distribution), and censorship resistance simultaneously, at acceptable latency? Order-fairness protocols (Aequitas lineage, Themis) trade throughput; encrypted mempools (threshold/witness encryption) trade latency and add cryptographic liveness assumptions. No production system yet delivers all three.
2. **Atomic cross-rollup composability.** Shared sequencing gives atomic *inclusion*, not atomic *execution* (which requires knowing state at build time). Can synchronous composability across sovereign execution environments be recovered without merging them? Based rollups + real-time proving is the most credible current path; a rigorous impossibility/possibility boundary is not yet established.
3. **Proof-system assurance.** Formal verification of full zkEVM circuits and zkVM compilers end-to-end; quantifying residual soundness risk; economic design of multi-prover regimes (how many, how independent, who pays). Related: post-quantum migration of the entire stack (KZG → hash-based commitments; BLS → hash-based signatures) without capacity regression — the Lean Ethereum programme is the flagship attempt.
4. **Prover decentralisation.** Real-time proving currently implies GPU clusters; ASICs are coming. Does proving centralise into an oligopoly, and does that matter (proofs are verifiable regardless — but liveness, censorship and pricing power remain)? Ethereum's home-prover targets (≤ 10 kW) are a deliberate research constraint; whether they hold under 1-gigagas blocks is open.
5. **DAS under network adversaries.** Formal guarantees for sampling under eclipse/Sybil attack, sampler anonymity (private information retrieval is too heavy; what is light enough?), and incentive-compatible reconstruction remain partially solved. Full Danksharding's 2D sampling and peer-scoring designs are still being specified.
6. **State growth.** No major general-purpose chain has shipped state expiry/rent for existing state. Can expiry, statelessness witnesses, and ZK-proved state access be combined into a design that bounds verifier state permanently without breaking contracts? (The binary-tree + zkEVM path assumes yes; it is not yet demonstrated at mainnet scale.)
7. **Interoperability security economics.** Intent/solver markets concentrate: what prevents solver oligopoly rent extraction, and how do we price the settlement risk solvers carry? More broadly: can bridge security be *measured* (a "value-at-risk per route" standard) rather than narrated?
8. **The verifier's dilemma, quantified.** For optimistic systems: what minimum challenger incentive provably sustains watching? For DAS: what sustains honest sampling at scale? Cryptoeconomic security proofs lag the deployed systems by years.
9. **Sustainable L2 economics.** If based/native designs return sequencing revenue to L1 and blob oversupply keeps DA revenue near zero, what funds L2 development and L1 security long-term? The interaction between scaling success and fee-burn-based security budgets is an unresolved macro-question for Ethereum in particular.

10. **Privacy at scale.** Compliance-compatible private transfers (ZK disclosure, viewing keys) on high-throughput rails — required by the TradFi workloads of Section 11.2, in tension with the transparency assumptions of today’s fraud proofs, MEV markets, and analytics-based security monitoring.

14. Summary

This chapter developed blockchain scalability from first principles to the production frontier. The core problem is structural: total replication buys trust-minimisation at the price of single-node throughput, and naive fixes (bigger blocks) spend decentralisation to buy capacity — the scalability trilemma. The escape consists of two primitives that make verification exponentially cheaper than execution: succinct proofs (fraud and validity) and data availability sampling. Layer 1 techniques — parameter increases earned by structural work (ePBS, BALs), consensus redesign (Alpenglow), parallel execution (Block-STM, Sealevel), sharding (execution sharding in NEAR/Polkadot; Ethereum’s pivot to data sharding), and state management (statelessness, expiry, binary trees) — raise the base layer’s ceiling. Layer 2 techniques move execution off-chain under on-chain enforcement: channels (instant, narrow), sidechains (sovereign, trusted), plasma (the instructive failure that isolated the DA problem), and rollups — optimistic (bisection games, BoLD, permissionless fault proofs at Stage 1) and ZK (zkEVM → zkVM industrialisation, real-time proving, the L1-zkEVM and native-rollup endgame). Data availability, the load-bearing concept, is now served by a priced spectrum from Ethereum blobs with PeerDAS through Celestia/Avail/EigenDA to committees, and the architect’s central act is choosing a point on that spectrum with open eyes. Security experience is unambiguous: attackers route around cryptography — through bridges, keys, committees, upgrade paths, and liveness gaps — and the defence stack (verification-path minimalism, rate limits, councils-with-timelocks, multi-provers, forced inclusion) is the hard-won response. DeFi turns performance failures into solvency failures and now drives scaling design; TradFi adoption made scalability a regulatory-grade requirement. No architecture has won — modular, integrated, and app-specific designs all hold ground — but all are converging on the same primitives, and the open problems of Section 13 mark where the next decade of research will be fought.

Key terms

Blob — a ~128 KB data package attached to Ethereum beacon blocks (EIP-4844), priced by an independent fee market and pruned after ~18 days; the rollup data-availability workhorse. **Blob-parameter-only (BPO) fork** — a pre-programmed, coordination-light fork that changes only blob count parameters (Fusaka). **Based rollup** — a rollup sequenced by the L1’s own proposers rather than a dedicated sequencer. **Data availability (DA)** — the guarantee that the data behind a commitment was published and is retrievable. **Data availability sampling (DAS)** — verifying DA probabilistically by requesting random erasure-coded chunks. **Escape hatch** — a code path allowing user exit against an uncooperative operator (forced inclusion, forced withdrawal, freeze-and-exit). **Fraud proof** — a proof that a posted state root is invalid, adjudicated on L1, typically via interactive bisection to a one-step proof. **Intent** — a signed declaration of desired outcome,

fulfilled competitively by solvers who carry execution and settlement risk (ERC-7683). **KZG commitment** — a constant-size polynomial commitment with constant-size opening proofs; binds blob data for DAS. **Multi-prover** — requiring agreement among independently implemented proof systems before accepting a state root. **Preconfirmation** — a bonded, slashable promise of future inclusion, restoring fast UX to based/decentralised sequencing. **Sequencer** — the party ordering and soft-confirming L2 transactions. **Stage 0/1/2** — L2Beat’s rollup-maturity ladder based on proof permissionlessness and upgrade constraints. **State root** — a cryptographic commitment (Merkle-family) to an entire state, enabling compact proofs about it. **Validity proof** — a succinct proof (SNARK/STARK) that a state transition was executed correctly; makes invalid states unpostable. **Validium** — validity-proved execution with off-chain data; freeze risk replaces theft risk. **Verifier’s dilemma** — the incentive failure whereby rational verifiers stop checking work that is almost always honest. **zkVM** — a prover for a general instruction set (usually RISC-V), allowing unmodified programs — including EVM clients — to be proved.

Further reading

Primary sources, in rough order of appearance in the chapter: Nakamoto, *Bitcoin: A Peer-to-Peer Electronic Cash System* (2008); Croman et al., *On Scaling Decentralized Blockchains* (FC 2016); Buterin, *A rollup-centric ethereum roadmap* (ethereum-magicians, 2020) and *An Incomplete Guide to Rollups* (vitalik.eth.limo, 2021); Poon & Dryja, *The Bitcoin Lightning Network* (2016); Poon & Buterin, *Plasma* (2017); Kalodner et al., *Arbitrum: Scalable, private smart contracts* (USENIX Security 2018); Luu et al., *Elastico* (CCS 2016); Kokoris-Kogias et al., *OmniLedger* (S&P 2018); Zamani et al., *RapidChain* (CCS 2018); Al-Bassam, Sonnino & Buterin, *Fraud and Data Availability Proofs* (2018); Kate, Zaverucha & Goldberg, *Constant-Size Commitments to Polynomials* (ASIACRYPT 2010); EIP-4844, EIP-7732, EIP-7928, EIP-7683 (eips.ethereum.org); the Fusaka/PeerDAS and Glamsterdam specifications (ethereum.org/roadmap); the Lean Ethereum roadmap (Ethereum Foundation blog, 2026); L2Beat (l2beat.com) for the stages framework and live risk analyses; Yakovenko, *Solana: A new architecture* (2018) and the Alpenglow paper (Anza, 2025); Gelashvili et al., *Block-STM* (PPoPP 2023); the Celestia (LazyLedger, Al-Bassam 2019), Avail and EigenDA documentation; Taiko’s based-rollup and preconfirmation posts; and the Arbitrum BoLD paper (Offchain Labs, 2024). Post-mortems worth assigning in full: Ronin (Sky Mavis, 2022), Wormhole (Jump/Certus, 2022), Nomad (2022), and the Base outage reports (2025–2026). For the institutional material of Section 11.3: the Canton Network whitepaper and Global Synchronizer documentation (canton.network); Deutsche Bank’s DAMA 2 litepaper (db.com, June 2025); Kinexys/J.P. Morgan’s JPM Coin announcements (jpmorgan.com); Finality’s regulatory filings and press archive (finality.com); SWIFT’s shared-ledger press releases (swift.com, 2026); and EY’s Nightfall_4 repository and release notes.

Further viewing: a video history of blockchain scalability

An unusual property of this field is that nearly its entire intellectual history is on public video. The debates happened at recorded workshops, the designs were first explained by their authors at recorded conferences, and the running commentary of the research

community lives in recorded podcasts. For students, this is a primary-source archive of remarkable quality: you can watch the block-size war being argued in the room, watch danksharding being proposed by the researcher it is named after, and watch the based-rollup thesis defended by its author — then check each claim against how events unfolded. The following curation is organised to track the chapter’s structure. Because video URLs rot, entries are cited by *title, speaker/channel, and year*; all are locatable by search on YouTube or the archives named below, and the standing archives — **archive.devcon.org** (every Devcon talk since 2014, indexed by playlist), **scalingbitcoin.org/presentations** (with transcripts at btctranscripts.com), and the channels of the Ethereum Foundation, StarkWare, Celestia, and the major podcasts — are the durable entry points.

Era 1 — Foundations and the block-size war (2015–2017); watch alongside Sections 1 and 2.1.

Resource	Source, year	Relevance
<i>Scaling Bitcoin</i> workshop recordings (Montreal, Hong Kong; Milan 2016)	scalingbitcoin.org / YouTube, 2015–2016	The block-size debate argued live by its protagonists; the venue where SegWit and the off-chain scaling consensus formed. The Hong Kong sessions are the primary record of Section 1’s governance lesson.
<i>Scalability and the Lightning Network</i> , Joseph Poon & Tadge Dryja	Epicenter (EB80), 2015	The Lightning authors explaining the channel construction of Section 3.1 in their own words, pre-launch.
<i>Blockchain and Money</i> , Prof. Gary Gensler	MIT OpenCourseWare (15.S12), 2018	A full university course on the foundations this chapter assumes; the scalability trade-off discussions are dated but the ledger fundamentals are timeless.
Andreas Antonopoulos’s scaling and Lightning lectures	aantonop channel, 2016–2019	The most accessible technically-sound explanations of the era; useful for students needing an on-ramp.

Era 2 — Plasma, early L2s, and the pivot to rollups (2017–2021); watch alongside Sections 3.3–3.4.

Resource	Source, year	Relevance
Devcon 3–5 Plasma and state-channel talks (Poon, Buterin, Fichter and others)	archive.devcon.org, 2017–2019	The rise and quiet abandonment of Plasma, on camera — the DA problem of Section 4.1 being discovered in public.
Barry Whitehat’s early zk-rollup presentations	Devcon 5 / ZK study groups, 2018–2019	The original “roll_up” construction that seeded the entire validity-proof scaling line.
<i>Endgame</i> , Vitalik Buterin	Bankless (ep. 99), 2022	The rollup-centric roadmap and its

interview		decentralisation reasoning (Sections 2.4, 8) explained by its author; pairs with reading the 2020 roadmap post.
Optimism and Arbitrum launch-era technical interviews (Kalodner, Goldfeder; Floersch)	Epicenter, Zero Knowledge, ETHGlobal workshops, 2020–2021	Fraud-proof design debates of Section 3.4.1 from the teams that shipped them.

Era 3 — Data availability and danksharding (2021–2026); watch alongside Section 4.

Resource	Source, year	Relevance
<i>Dive into Danksharding</i> , Buterin, Feist, Protolambda, mod. Beiko	Bankless, 2022	The canonical accessible explanation of danksharding and blobs by its designers.
<i>Danksharding Workshop</i> , EF Consensus R&D	Devcon 6 archive, 2022	The technical version: DAS, KZG, and the 2D scheme of Section 4.3 at whiteboard depth.
<i>The KZG Ceremony — or How I Learnt to Stop Worrying and Love Trusted Setups</i>	Devcon 6 archive, 2022	Section 4.2’s trusted-setup discussion, from the ceremony’s organisers.
Mustafa Al-Bassam’s modular blockchain talks and interviews	Celestia channel; Delphi <i>DISRUPTORS</i> ; CLG Podcast, 2022–2025	The LazyLedger-to-Celestia thesis (Section 4.4) argued by its author — the strongest statement of the modular position.
PEEPanEIP series (EIP-4844 episodes)	Ethereum Cat Herders channel, 2022–2023	Implementer-level walkthroughs of the blob mechanics of Section 3.5.

Era 4 — ZK proof systems and zkVMs (2018–2026); watch alongside Section 5.

Resource	Source, year	Relevance
<i>The Journey of STARK Technology</i> , Eli Ben-Sasson	StarkWare Sessions, 2023	STARKs from their co-inventor; StarkWare’s channel also carries the <i>STARK math</i> lecture series behind Section 5.2.
<i>Hasu gets STARK-pilled</i> , Eli Ben-Sasson	Uncommon Core 2.0, 2021	Deliberately built as “the most approachable comprehensive audio resource on how STARKs work”; it largely succeeds.
Zero Knowledge podcast zkEVM arc — eps. 175/176 (Matter Labs, Aztec), 194 (Hermez zkEVM), 210 (Miden), later	zeroknowledge.fm, 2021–2026	The industrialisation of proving (Section 5.3) documented in real time across five years of researcher interviews —

Ethproofs/zkVM episodes with Justin Drake

zkEVM formal-verification and real-time-proving talks

EF zkEVM project, ZK Summit recordings, 2025–2026

listened to in sequence, it is a course in itself.

The current frontier: multi-provers, RISC-V zkVMs, and the L1-zkEVM programme of Section 8.

Era 5 — Sequencing, based rollups, and the modern roadmap (2023–2026); watch alongside Sections 6–8.

Resource	Source, year	Relevance
<i>Do Based Rollups Solve Everything?</i> , Justin Drake	Bankless, 2024	The based-sequencing thesis of Section 6 by its author; pair with his earlier shared-sequencing episode (<i>The United Rollups of Ethereum</i> , with Ben Fisch).
<i>Restaking Alignment</i> , Buterin, Kannan, Beiko, Drake, Feist	Bankless (video), 2024	The restaking/EigenDA trust-assumption debate of Sections 4.4 and 9.5, with the principals disagreeing on camera.
Devcon SEA and Devconnect keynotes and Surge-track talks	archive.devcon.org, 2024–2026	The current roadmap (PeerDAS, ePBS, FOCIL, Lean Ethereum) presented by the researchers delivering it.
Solana Breakpoint sessions (Firedancer demonstrations; Alpenglow presentations)	Solana Foundation channel, 2022–2026	The integrated-chain counter-thesis of Section 8, including the laboratory 1M-TPS demonstrations. Section 8 teaches students to read critically.
<i>Ethereum Engineering Group</i> technical seminars (rollup economics, EIP-4844, L2 architecture series)	Ethereum Engineering Group channel, 2021–2026	Long-form engineering seminars — closer to a graduate reading group than to conference talks; the rollup-economics sessions complement Section 3.5.

Standing series worth following (the field moves; these are the feeds through which it moves on video): the Devcon/Devconnect archive playlists; the Ethereum Foundation’s *Protocol* calls and All Core Devs recordings (the unedited primary record of every fork discussed in this chapter, including Fusaka and Glamsterdam); Zero Knowledge podcast; Bankless’s technical episodes; StarkWare Sessions; ETHGlobal’s workshop archive; Celestia’s *Modular Summit* recordings; and L2BEAT’s governance and risk video series. A practical note for instructors: conference talks age into historical documents rather than references — assign Era 1–3 videos as *history* to be checked against Sections 1–4, and only Era 5 material as current-state description, applying the same dating discipline the chapter applies to its own numbers.

Review questions

1. State the scalability trilemma precisely, and explain why it constrains naive designs but not stratified ones. Which two primitives enable the stratification?
2. Why is 20,000 TPS in a laboratory not 20,000 TPS on mainnet? Give three distinct reasons drawn from the node-resource model.
3. Explain why data withholding is not attributable, and why this forces DA to be solved by sampling rather than fraud proofs. Derive the $2^{(-k)}$ detection bound.
4. A fraud-proof window is 7 days; an upgrade timelock is 8 days; the security council can act instantly on a “provable bug.” Which L2Beat stage is this, and what attack does each parameter defend against?
5. Classify the Ronin, Wormhole, and Nomad exploits within the four-part bridge-attack taxonomy, and name the single defence that would most cheaply have limited each.
6. Your team is launching a perp DEX targeting 5,000 TPS. Argue for a DA choice (blobs, Celestia, EigenDA, DAC) and a sequencing choice (centralised, decentralised, based), stating the trust assumptions you are accepting and how you would disclose them to users.
7. Modify the chapter’s Python program so the DAS adversary can eclipse the sampler (answering its queries selectively while withholding from others). What guarantee survives, and what additional mechanism restores security?
8. Why might a ZK rollup soundness bug be strictly worse than an optimistic rollup fraud-proof failure? What deployed and planned mechanisms mitigate it?
9. Explain how EIP-4844’s independent blob fee market decouples L2 costs from L1 congestion, and why BPO forks were introduced in Fusaka rather than simply setting the blob maximum to 48 immediately.
10. Take a position, with technical argument: will atomic cross-rollup composability be recovered, or is asynchronous intent-based interoperability the permanent end state?
11. Using the five questions of the “How to read a TPS claim” discussion, critically evaluate one published throughput claim from any chain’s documentation, identifying which of the five it answers and which it evades.
12. (Programming.) Extend the chapter’s rollup demonstration with a deposit/withdrawal bridge contract model: an L1 balance map, a deposit function crediting L2, and a withdrawal that succeeds only after either (a) a modelled 7-day window or (b) a validity flag set by re-execution. Then implement the attack: let the proposer post a false root during the window and show which invariant the fraud-proof path must protect for the bridge to remain solvent.